

Machine Learning Workshop | L01

Introduction to Machine Learning

Background

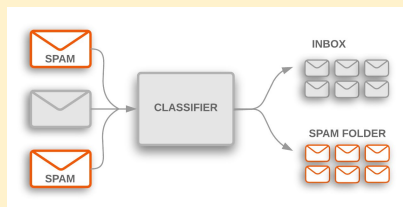
Artificial Intelligence

Any techniques that enables computers to mimic human behaviour



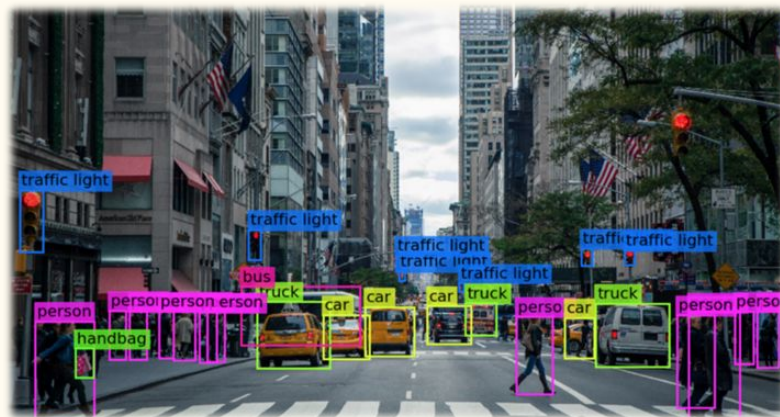
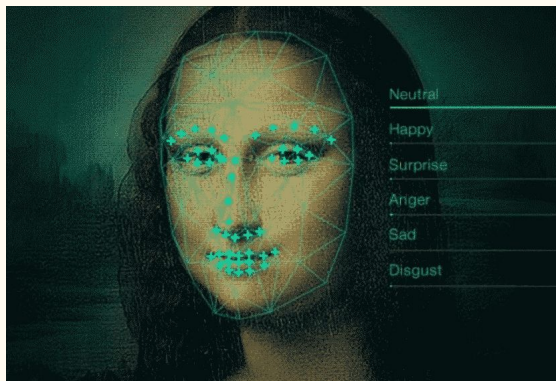
Machine Learning

Ability to learn without explicitly being programmed

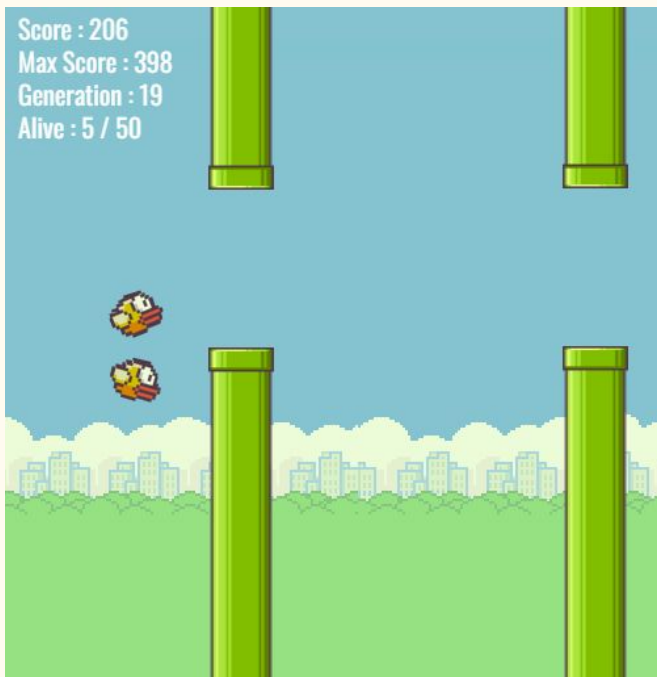


Examples in our everyday life

- Human Language and Communication (NLP Natural Language Processing)
 - e.g. AI Chatbot, Translator
- Computer Vision
 - e.g . Facial recognition, Self-driving car



How a machine learn to play a game



<https://xviniette.github.io/FlappyLearning/>

```
Network.prototype.getSave = function () {  
  var datas = {  
    neurons: [], // Number of Neurons per layer.  
    weights: [] // Weights of each Neuron's inputs.  
  };  
  
  for (var i in this.layers) {  
    datas.neurons.push(this.layers[i].neurons.length);  
    for (var j in this.layers[i].neurons) {  
      for (var k in this.layers[i].neurons[j].weights) {  
        // push all input weights of each Neuron of each Layer into a flat  
        // array.  
        datas.weights.push(this.layers[i].neurons[j].weights[k]);  
      }  
    }  
  }  
  return datas;  
}
```

Python?



- Python code is understandable by humans, which is easy to learn
- Wide community and free access to great libraries and frameworks for AI and ML



Python?



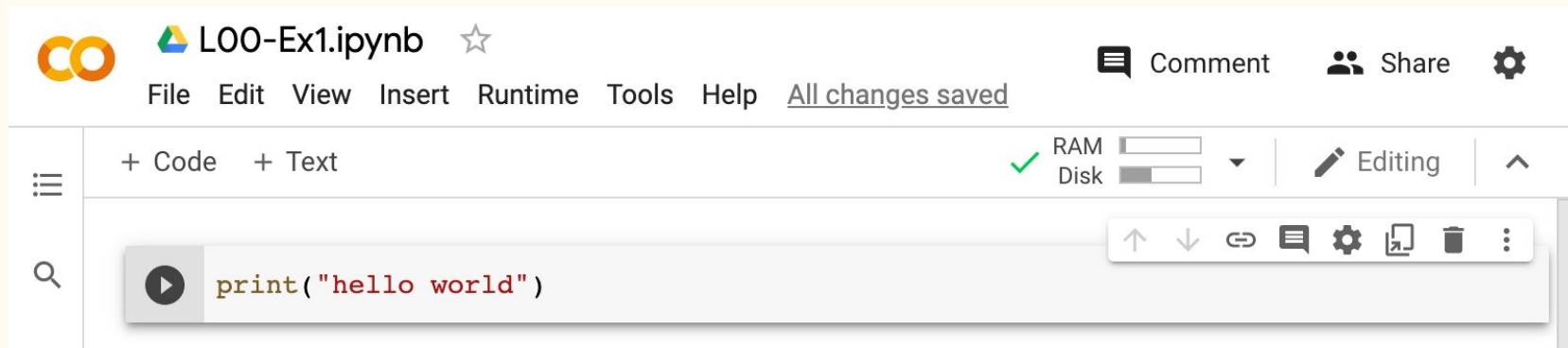
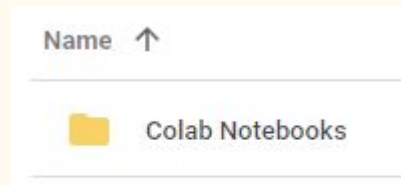
- Python code is understandable by humans, which is easy to learn
- Wide community and free access to great libraries and frameworks for AI and ML



:) There is no such thing as a “best language for machine learning”

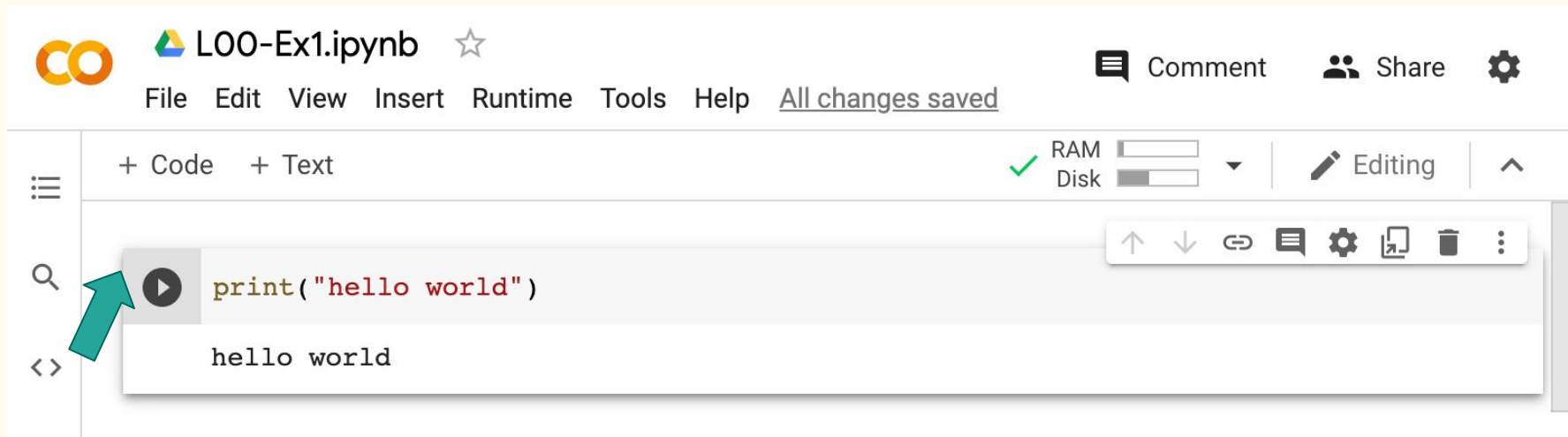
Google Colab - Hello World!

- You will need a google account
- <https://colab.research.google.com/>
- File → New notebook
- Change the filename to **L01_ex1.ipynb**
- Type this in the first cell:
 - `print("hello world")`



Execute Python on Google CoLab

- Tap the “play”  button next to the cell to execute (or pressing CTRL+Enter)





Example:

Detecting spam messages

Detecting spam messages

If you can read all my old emails, what sequence of instructions would you explicitly programme to automatically detect spam among newly received emails?



Mayana Bamde <eftfca@gmail.com>

From Miss Mayana Bamde.

Good day, I hope all is well. It is my pleasure to contact you for a business venture which I intend to establish with you in your country, though we have not met before, but one You have to believe in someone to succeed.

I'm 20 years old and the only daughter of my father, Late Mr & Mrs Jean Bamde, Native of Odienné republic of Ivory Coast or (Cote D'Ivoire). Because of the ongoing civil war series in my country for several years ago, there is this huge amount of eight million, five hundred thousand US dollars (US \$ 8.500,000.00) which my late father deposited in finance firm Ivory Coast here before he was murdered by some unknown enemies. Now I decided to invest this money in your country or anywhere safe place enough for security and political reasons.

Maria Saoy <yrkjr529@sepia.plala.or.jp>

My Dear Friend,

I am Mrs. Maria Am an aging widow suffering from a long time brain tumor disease, I am currently admitted in a private hospital here in Canada. I have decided to hand over and entrust the sum of (\$7 Million Canadian Dollar) in my account, and I need an honest and God fearing person who can use this fund for orphanage homes and 40% of the fund will be your compensation for doing this work of God.

Please if you volunteer to carry out this work for the said purpose kindly reply to me so that you can know more about me and how to achieve this project. Please your religion does not matter to me, what matters most is using the fund with the fear of Almighty God.

I wait to hear from you

Thanks
Mrs. Maria

Detecting spam messages

Compare sender email address against list of email addresses I've responded to before:

default email.class is 0 i.e. unknown

for email in new_email:

if email.sender not in respond_list:

email.class = 1 *# spam!*

else:

email.class = 2 *# not spam*

Mayana Bamde <eftca@gmail.com>

From Miss Mayana Bamde.

Good day, I hope all is well. It is my pleasure to contact you for a business venture which I intend to establish with you in your country, though we have not met before, but one You have to believe in someone to succeed.

I'm 20 years old and the only daughter of my father, Late Mr & Mrs Jean Bamde, Native of Odienné republic of Ivory Coast or (Cote D'Ivoire). Because of the ongoing civil war series in my country for several years ago, there is this huge amount of eight million, five hundred thousand US dollars (US \$ 8.500,000.00) which my late father deposited in finance firm Ivory Coast here before he was murdered by some unknown enemies. Now I decided to invest this money in your country or anywhere safe place enough for security and political reasons.

Maria Saoy <yrkjr529@sepia.plala.or.jp>

My Dear Friend,

I am Mrs. Maria Am an aging widow suffering from a long time brain tumor disease, I am currently admitted in a private hospital here in Canada. I have decided to hand over and entrust the sum of (\$7 Million Canadian Dollar) in my account, and I need an honest and God fearing person who can use this fund for orphanage homes and 40% of the fund will be your compensation for doing this work of God.

Please if you volunteer to carry out this work for the said purpose kindly reply to me so that you can know more about me and how to achieve this project. Please your religion does not matter to me, what matters most is using the fund with the fear of Almighty God.

I wait to hear from you

Thanks
Mrs. Maria

Detecting spam messages

Compare sender email address against list of email addresses I've responded to before:

default email.class is 0 i.e. unknown

for email in new_email:

if email.sender in respond_list:

email.class = 2 *# not spam*

Mayana Bamde <eftfca@gmail.com>

From Miss Mayana Bamde.

Good day, I hope all is well. It is my pleasure to contact you for a business venture which I intend to establish with you in your country, though we have not met before, but one You have to believe in someone to succeed.

I'm 20 years old and the only daughter of my father, Late Mr & Mrs Jean Bamde, Native of Odienné republic of Ivory Coast or (Cote D'Ivoire). Because of the ongoing civil war series in my country for several years ago, there is this huge amount of eight million, five hundred thousand US dollars (US \$ 8.500.000.00) which my late father deposited in finance firm Ivory Coast here before he was murdered by some unknown enemies. Now I decided to invest this money in your country or anywhere safe place enough for security and political reasons.

Maria Saoy <yrkjr529@sepia.plala.or.jp>

My Dear Friend,

I am Mrs. Maria Am an aging widow suffering from a long time brain tumor disease, I am currently admitted in a private hospital here in Canada. I have decided to hand over and entrust the sum of (\$7 Million Canadian Dollar) in my account, and I need an honest and God fearing person who can use this fund for orphanage homes and 40% of the fund will be your compensation for doing this work of God.

Please if you volunteer to carry out this work for the said purpose kindly reply to me so that you can know more about me and how to achieve this project. Please your religion does not matter to me, what matters most is using the fund with the fear of Almighty God.

I wait to hear from you

Thanks
Mrs. Maria

Detecting spam messages

Spam: email from someone I have never communicated with before that talks about money

default email.class is 0 i.e. unknown

for email in new_email:

if email.sender in respond_list:

email.class = 2 *# not spam*

else:

if "\$" in email.text:

email.class = 1 *# spam!*

Mayana Bamde <eftfca@gmail.com>

From Miss Mayana Bamde.

Good day, I hope all is well. It is my pleasure to contact you for a business venture which I intend to establish with you in your country, though we have not met before, but one You have to believe in someone to succeed.

I'm 20 years old and the only daughter of my father, Late Mr & Mrs Jean Bamde, Native of Odienné republic of Ivory Coast or (Cote D'Ivoire). Because of the ongoing civil war series in my country for several years ago, there is this huge amount of eight million, five hundred thousand US dollars (US \$ 8.500.000.00) which my late father deposited in finance firm Ivory Coast here before he was murdered by some unknown enemies. Now I decided to invest this money in your country or anywhere safe place enough for security and political reasons.

Maria Saoy <yrkjr529@sepia.plala.or.jp>

My Dear Friend,

I am Mrs. Maria Am an aging widow suffering from a long time brain tumor disease, I am currently admitted in a private hospital here in Canada. I have decided to hand over and entrust the sum of (\$7 Million Canadian Dollar) in my account, and I need an honest and God fearing person who can use this fund for orphanage homes and 40% of the fund will be your compensation for doing this work of God.

Please if you volunteer to carry out this work for the said purpose kindly reply to me so that you can know more about me and how to achieve this project. Please your religion does not matter to me, what matters most is using the fund with the fear of Almighty God.

I wait to hear from you

Thanks
Mrs. Maria

Detecting spam messages

Spam: email from someone I have never communicated with before that talks about money

default email.class is 0 i.e. unknown

for email in new_email:

if email.sender in respond_list:

email.class = 2 *# not spam*

else:

if "\$" in email.text:

email.class = 1 *# spam!*

elif "dollar" in email.text:

email.class = 1 *# spam!*

Mayana Bamde <eftfca@gmail.com>

From Miss Mayana Bamde.

Good day, I hope all is well. It is my pleasure to contact you for a business venture which I intend to establish with you in your country, though we have not met before, but one You have to believe in someone to succeed.

I'm 20 years old and the only daughter of my father,Late Mr & Mrs Jean Bamde, Native of Odienné republic of Ivory Coast or (Cote D'Ivoire).Because of the ongoing civil war series in my country for several years ago,there is this huge amount of eight million, five hundred thousand US dollars (US \$ 8.500.000.00) which my late father deposited in finance firm Ivory Coast here before he was murdered by some unknown enemies. Now I decided to invest this money in your country or anywhere safe place enough for security and political reasons.

Maria Saoy <yrkjr529@sepia.plala.or.jp>

My Dear Friend,

I am Mrs. Maria Am an aging widow suffering from a long time brain tumor disease, I am currently admitted in a private hospital here in Canada. I have decided to hand over and entrust the sum of (\$7 Million Canadian Dollar) in my account, and I need an honest and God fearing person who can use this fund for orphanage homes and 40% of the fund will be your compensation for doing this work of God.

Please if you volunteer to carry out this work for the said purpose kindly reply to me so that you can know more about me and how to achieve this project. Please your religion does not matter to me, what matters most is using the fund with the fear of Almighty God.

I wait to hear from you

Thanks
Mrs. Maria

Detecting spam messages

default email.class is 0 i.e. unknown

for email **in** new_email:

if email.sender **in** respond_list:

 email.class = 2 *# not spam*

else:

if “\$” **in** email.text:

 email.class = 1 *# spam!*

elif “dollar” **in** email.text:

 email.class = 1 *# spam!*

New email

Send from anonymous

Talk about money
⇒ spam!

Detecting spam messages

★: Actual spam message

default email.class is 0 i.e. unknown

```
for email in new_email:
```

```
    if email.sender in respond_list:
```

```
        email.class = 2  # not spam
```

```
    else:
```

```
        if "$" in email.text:
```

```
            email.class = 1  # spam!
```

```
        elif "dollar" in email.text:
```

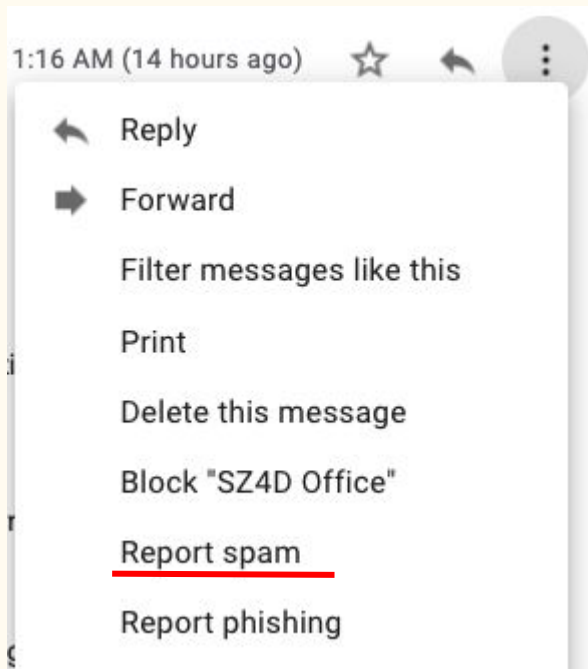
```
            email.class = 1  # spam!
```

New email

Send from anonymous

Talk about money
⇒ spam!

Learning from experience



- Generate labelled dataset of spam and non-spam emails.
- Start with a large number of features e.g. word count, most common word, time email was sent, number of times “please” appears, size of attachments etc.
- Identify common features of spam vs non-spam emails!

Binary tree classifier - training steps

1. For each axis (feature) j , find best splitting point t_j
2. Select best splitting axis i
3. Split along axis i at t_i .
Assign group class based on most common class.
4. Repeat n times

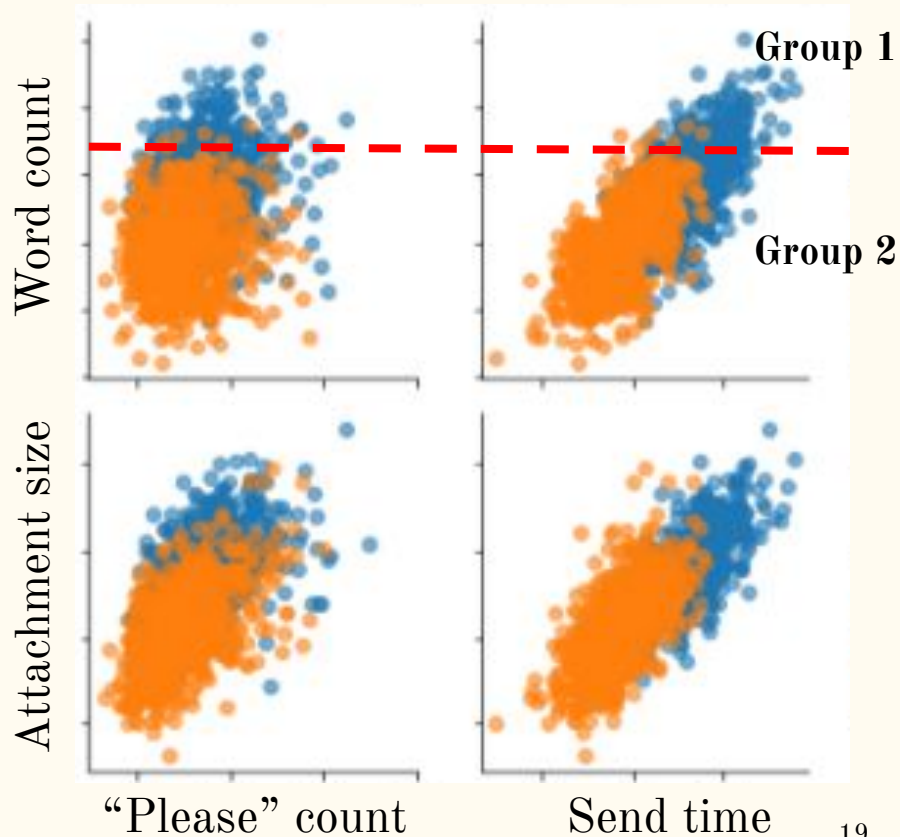
How to quantify “best”?

Smallest proportion of points are misclassified if we choose to split at t_i

Binary tree classifier

1. For each axis (feature) j , find best splitting point t_j
2. Select best splitting axis i
3. Split along axis i at t_i .
Assign group class based on most common class

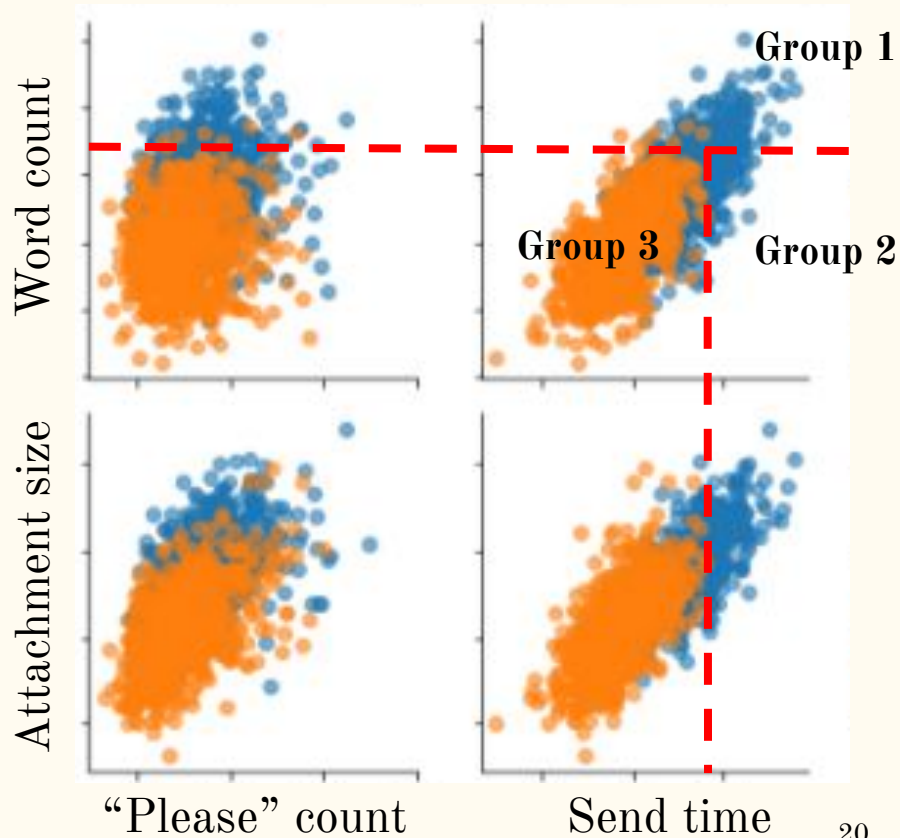
- Spam
- Not spam



Binary tree classifier

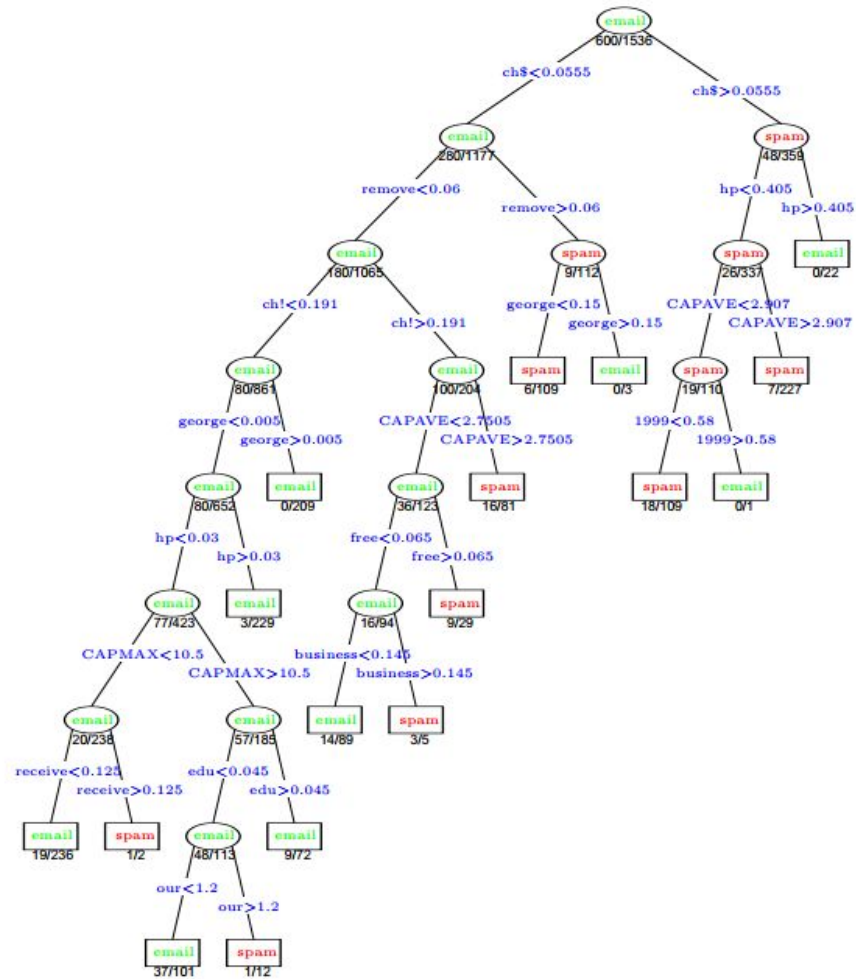
1. For each axis (feature) j , find best splitting point t_j
2. Select best splitting axis i
3. Split along axis i at t_i .
Assign group class based on most common class
4. Repeat n times (using other features)

- Spam
- Not spam



Binary tree classifier

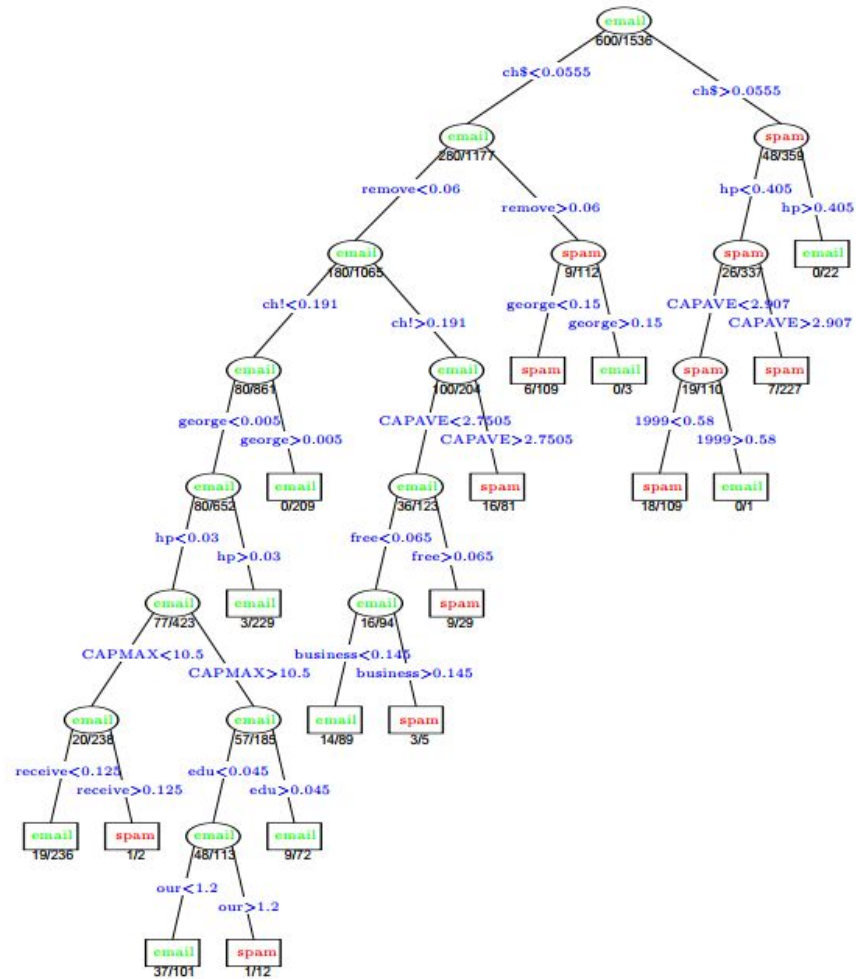
- Machine automatically learnt what features and thresholds to use to classify emails from experience!



Binary tree classifier

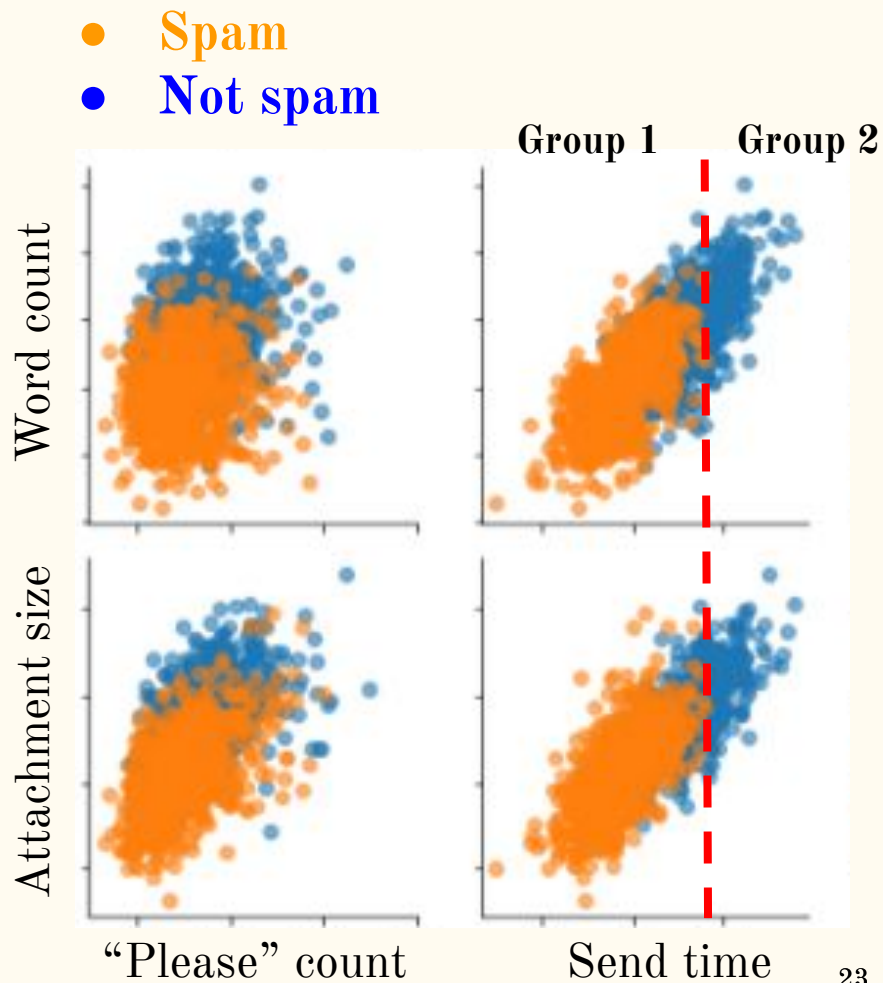
- Machine automatically learnt what features and thresholds to use to classify emails from experience!

4. Repeat n times



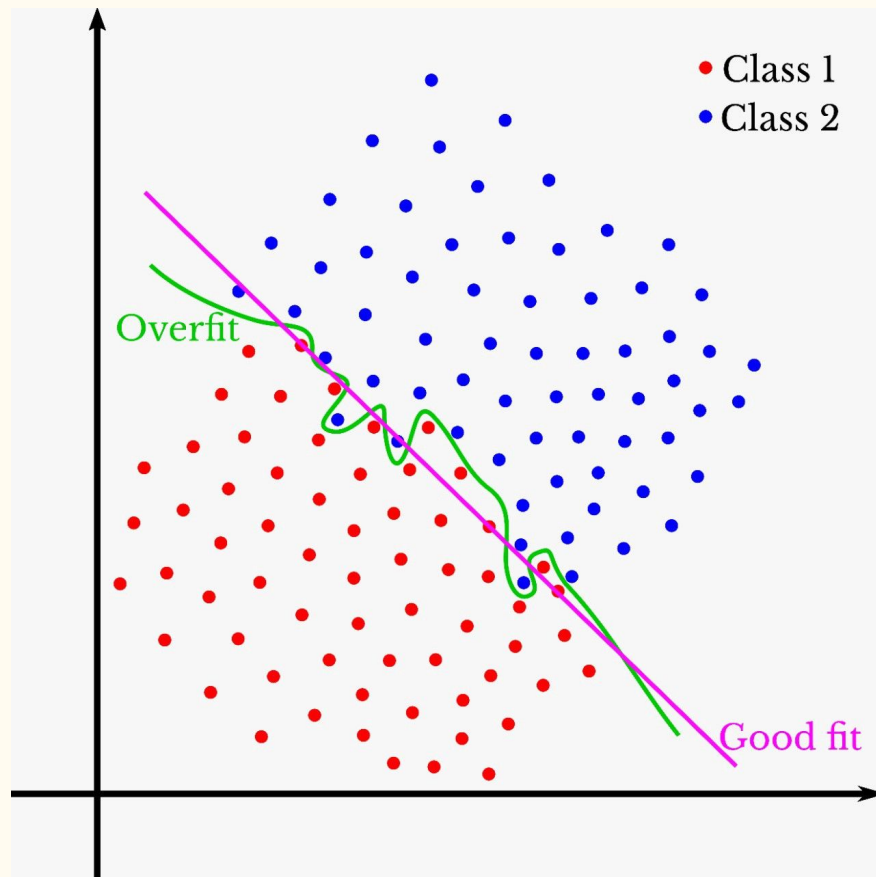
Influence of tree depth n

- n too small: Insufficient accuracy



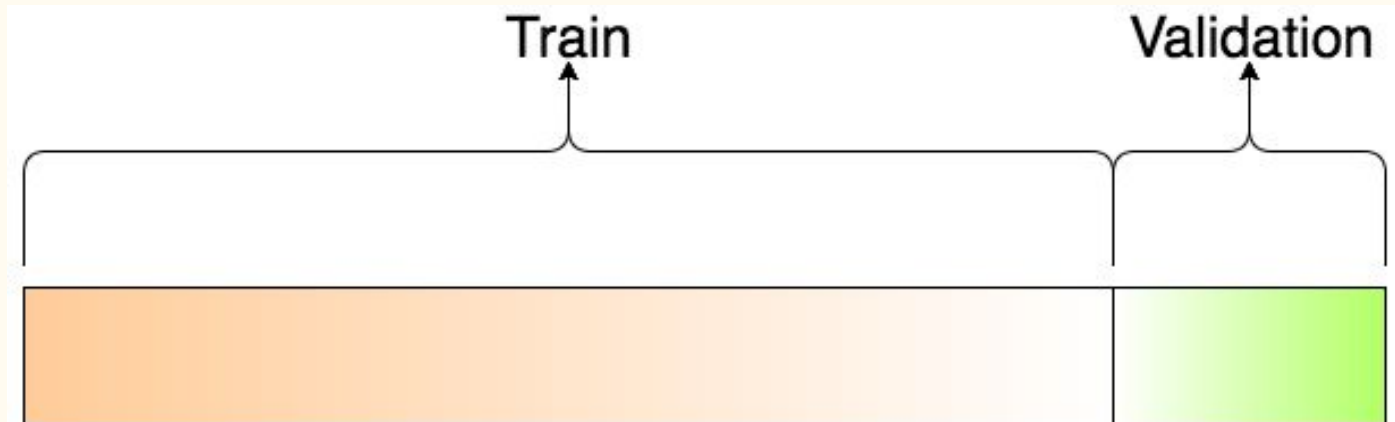
Influence of tree depth n

- n too small: Insufficient accuracy
- n too big: Overfitting

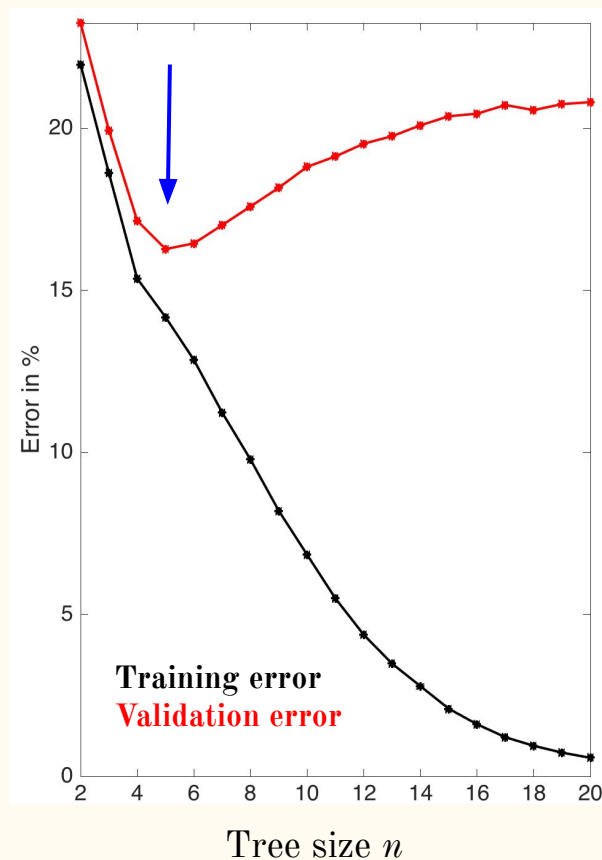


Training vs validation dataset

1. Train models with different n values on training dataset.
2. Apply trained models on validation dataset.
3. Choose model with best performance.



Influence of tree depth n



Training vs validation vs test dataset

1. Train models with different n values on training dataset.
2. Apply trained models on validation dataset.
3. Choose model with best performance.
4. Apply chosen model on test dataset and report results.



Summary: supervised machine learning

1. Obtain labelled dataset (e.g. basaltic vs granitic rocks) with list of features (e.g. density, color, hardness).
2. Split dataset into training, validation, and test sets.
3. Choose machine learning algorithm (e.g. decision tree).
4. Train machine learning model, using validation set to tune hyperparameters (e.g. tree depth).
5. Choose best model and report performance on test set.



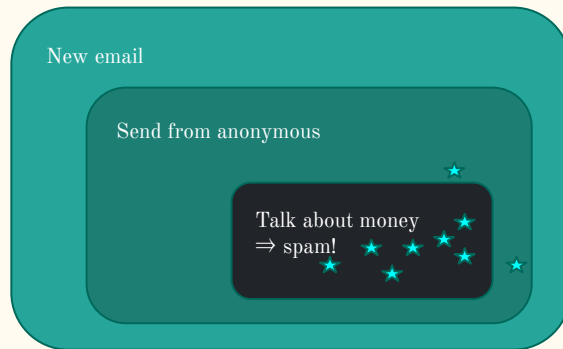
Machine learning

The big picture

Paradigms of algorithm - Different approaches

1) Rule-based approach

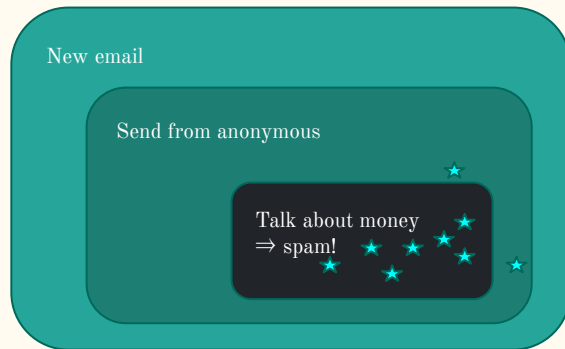
- Algorithms following the logic of cake recipe - coding line by line
- A set of fixed instructions and rules
- Deterministic



Paradigms of algorithm - Different approaches

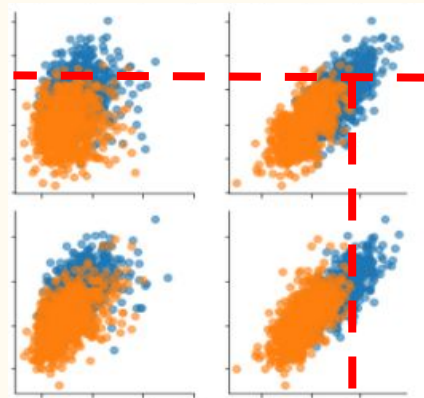
1) Rule-based approach

- Algorithms following the logic of cake recipe - coding line by line
- A set of fixed instructions and rules
- Deterministic



2) Machine learning approach

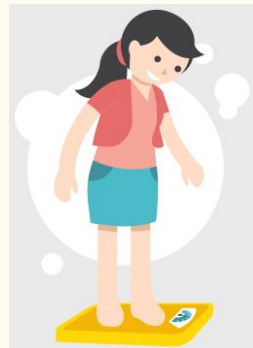
- Algorithms constantly evolve during training and learning processes
- Ability to learn and figure out the rules and features from the data, and for predicting outcomes
- Probabilistic



A Prediction

$$BMI = f(Weight, Height)$$

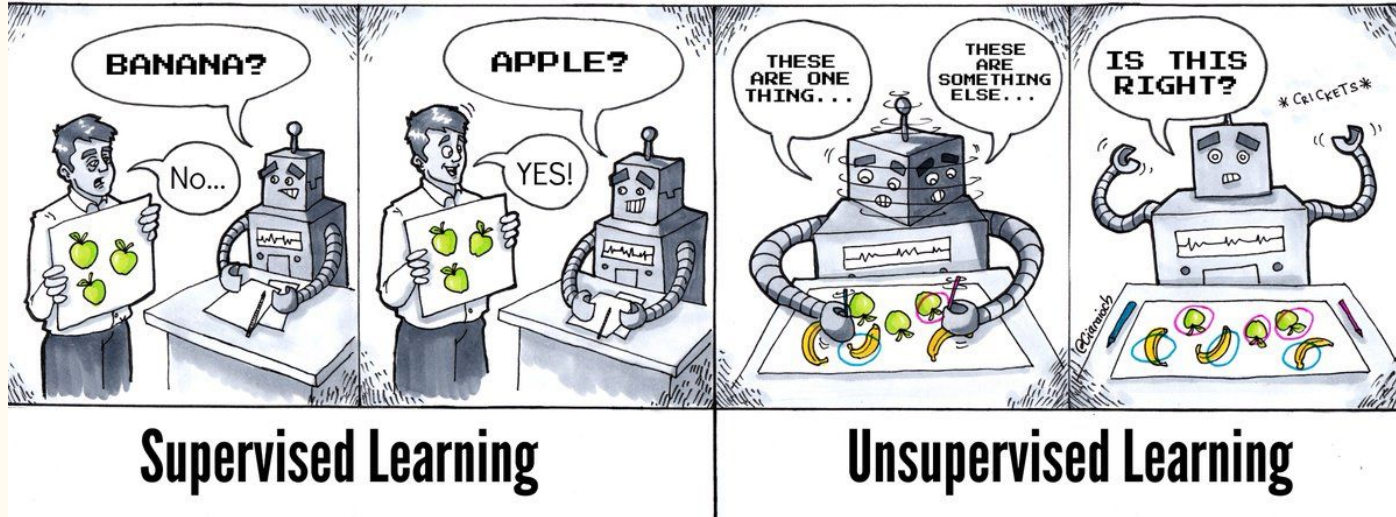
$$BMI = \frac{Weight(kg)}{[Height(m)]^2}$$



For **machine learning approach**, you are able to **discover the unknown relationship** of data, and for the future prediction!

$$y = f(x_1, x_2, x_3, \dots)$$

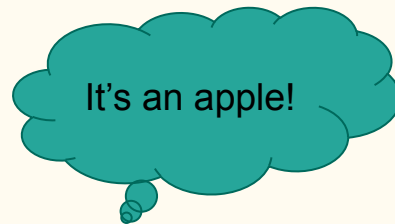
Types of machine learning



Types of machine learning

Supervised learning

labeled input data



- Identify their common patterns/trends in the data
- Discover the relationship between data

Algorithms

Unsupervised learning

unlabeled input data



Semi-supervised learning | Reinforcement learning

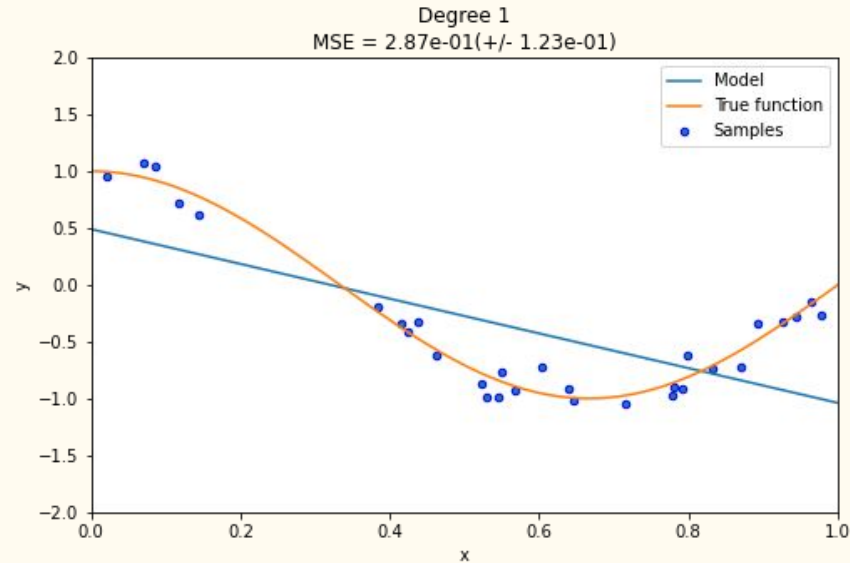
Major algorithms

Supervised
Unsupervised

}

Regression
Classification

Regression - to quantify relationship between two or more variables (i.e. X vs y)



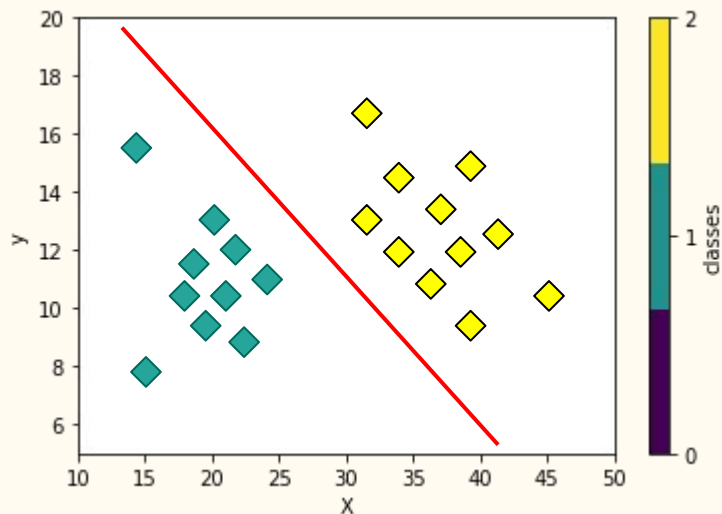
Major algorithms

Supervised
Unsupervised

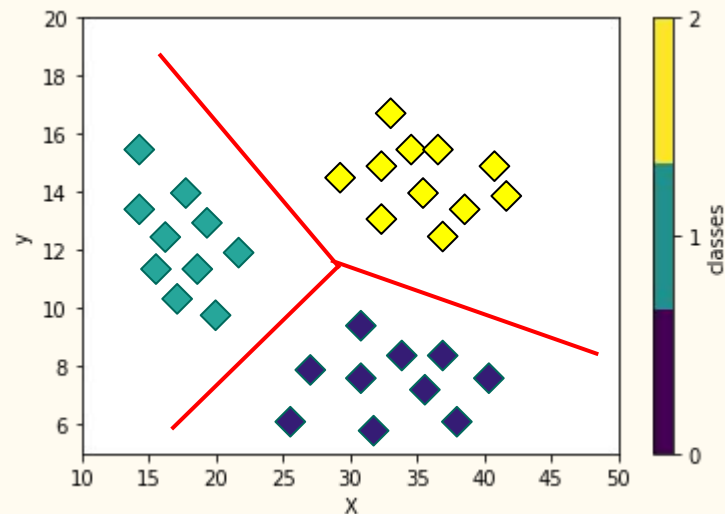
$\left\{ \begin{array}{l} \text{Regression} \\ \text{Classification} \end{array} \right.$

Classification - to categorize a given set of data into classes

Binary classification



Multi-class classification



Major algorithms

Supervised
Unsupervised { Clustering
...

Clustering - to find distinct groups or “clusters” within a data set, based on some similarity measures

unlabeled data

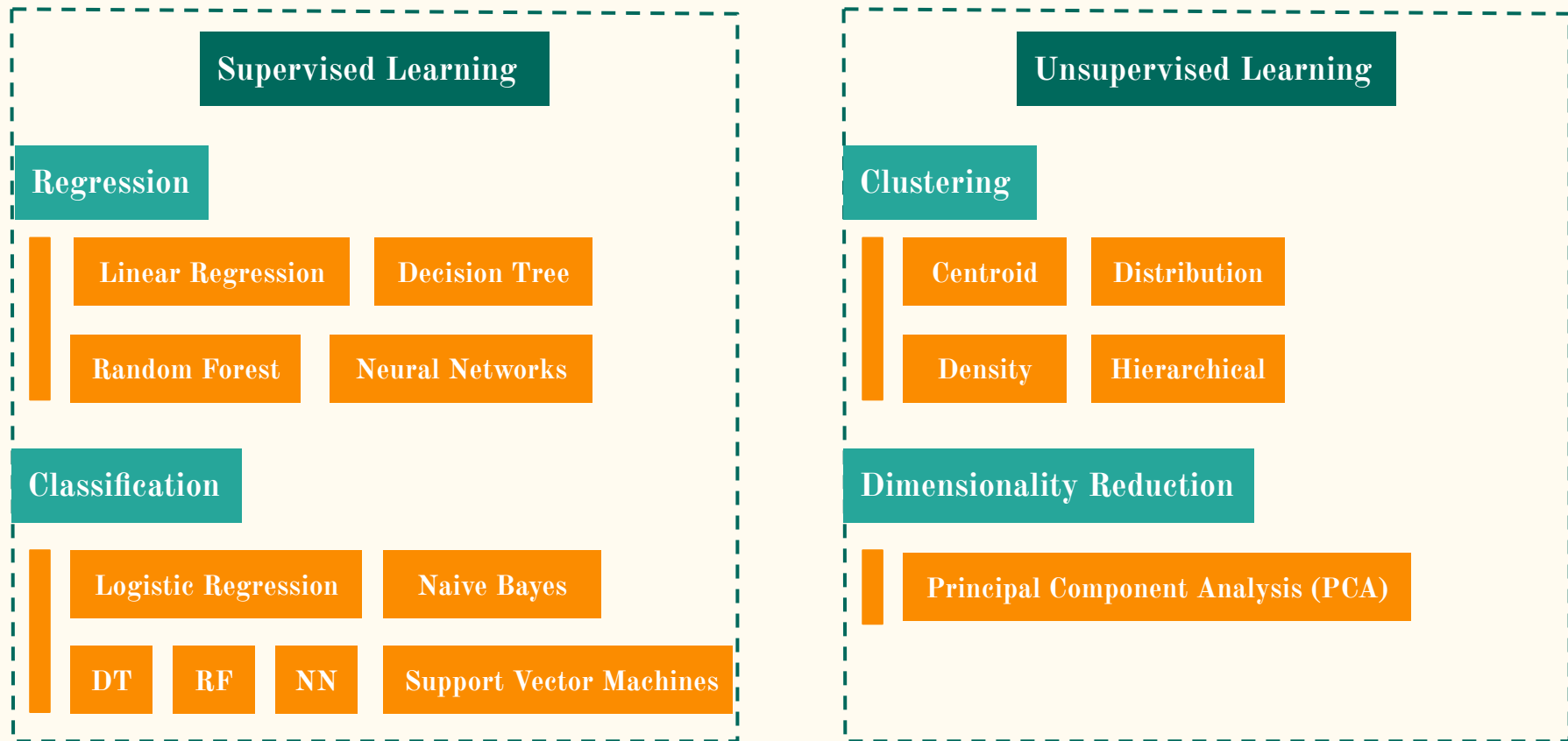


Algorithms

data cluster



Machine learning - the big picture



This workshop will cover :

Supervised Learning

Regression

Linear Regression

Decision Tree

Random Forest

Neural Networks

Classification

Logistic Regression

Naive Bayes

DT

RF

NN

Support Vector Machines

Unsupervised Learning

Clustering

Centroid

Distribution

Density

Hierarchical

Dimensionality Reduction

Principal Component Analysis (PCA)

Example - Classification

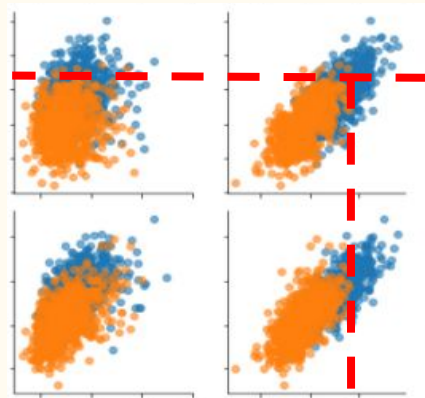
Iris flowers dataset

Binary tree classifier - training steps

1. For each axis (feature) j , find best splitting point t_j
2. Select best splitting axis i
3. Split along axis i at t_i .
Assign group class based on most common class.
4. Repeat n times

How to quantify “best”?

Smallest proportion of points are misclassified if we choose to split at t_i



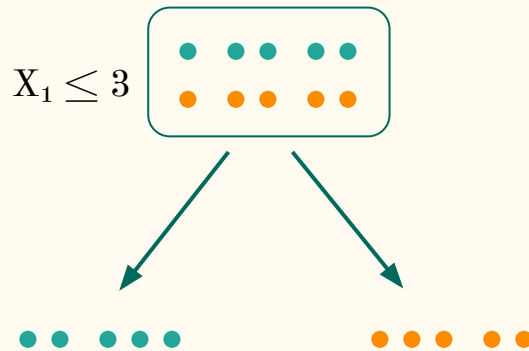
Gini Criterion

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

p_i : probability of class i

Gini impurity measures the quality of a split.
It varies between 0 and 1.

If all elements belong to a single class $\Rightarrow$ Pure = 0



Gini Criterion

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

p_i : probability of class i

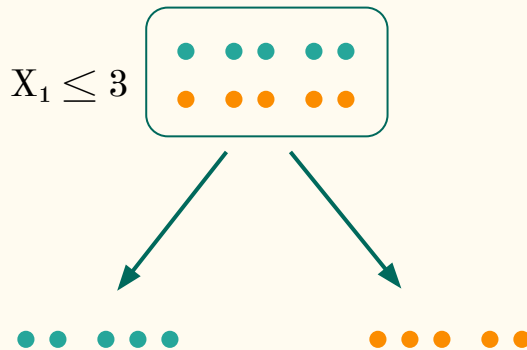
Gini impurity measures the quality of a split.
It varies between 0 and 1.

If all elements belong to a single class $\Rightarrow$ Pure = 0

$$P(\text{Green}) = 5/10$$

$$P(\text{Orange}) = 5/10$$

$$\begin{aligned} \text{Gini} &= 1 - [P(\text{Green})^2 + P(\text{Orange})^2] \\ &= 1 - (0.25 + 0.25) \\ &= 0.5 \end{aligned}$$



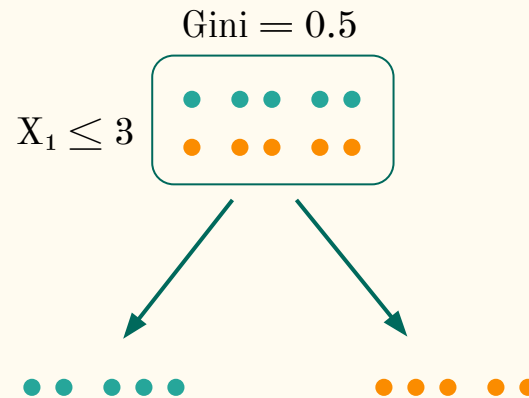
Gini Criterion

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

p_i : probability of class i

Gini impurity measures the quality of a split.
It varies between 0 and 1.

If all elements belong to a single class $\Rightarrow$ Pure = 0



$$P(\text{Green}) = 5/5$$

$$P(\text{Orange}) = 0/5$$

$$\begin{aligned} Gini &= 1 - [P(\text{Green})^2 + P(\text{Orange})^2] \\ &= 1 - (1 + 0) \\ &= 0 \end{aligned}$$

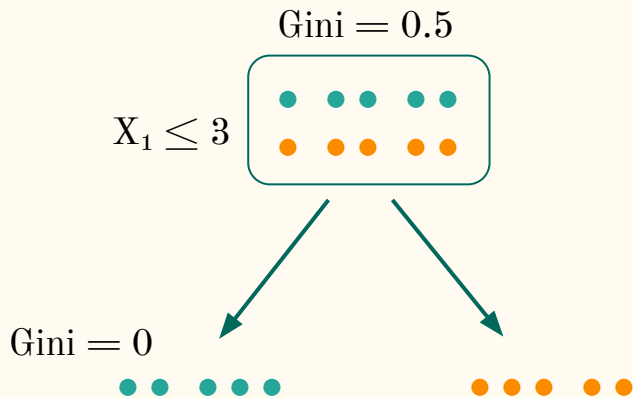
Gini Criterion

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

p_i : probability of class i

Gini impurity measures the quality of a split.
It varies between 0 and 1.

If all elements belong to a single class $\Rightarrow$ Pure = 0



$$P(\text{Green}) = 0/5$$

$$P(\text{Orange}) = 5/5$$

$$\begin{aligned} Gini &= 1 - [P(\text{Green})^2 + P(\text{Orange})^2] \\ &= 1 - (0 + 1) \\ &= 0 \end{aligned}$$

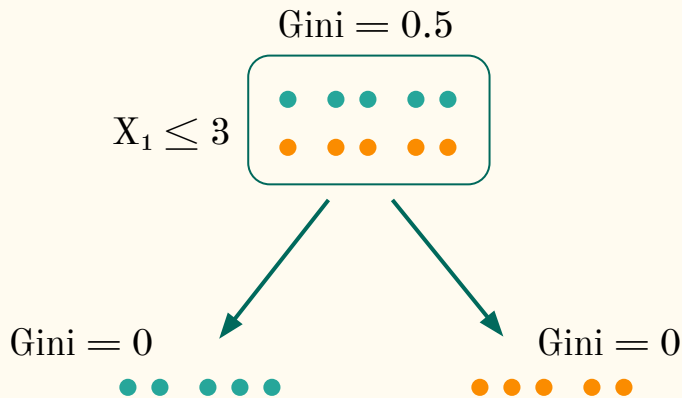
Gini Criterion

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

p_i : probability of class i

Gini impurity measures the quality of a split.
It varies between 0 and 1.

If all elements belong to a single class $\Rightarrow$ Pure = 0



Gini Criterion & Information Gain

Informarion Gain

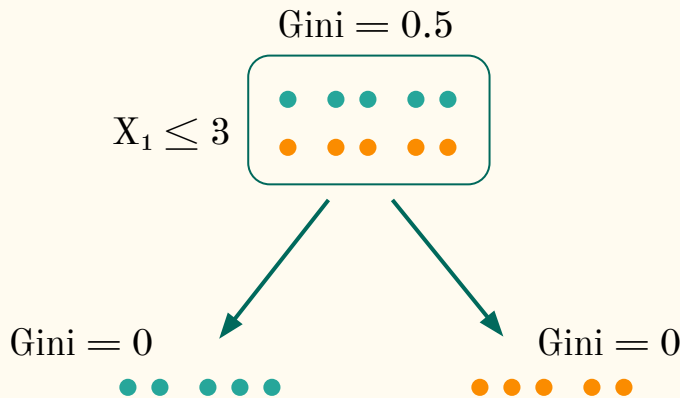
$$= \text{Information}(\text{parent}) - \text{Information}(\text{child})$$

$$= Gini(\text{parent}) - \sum \text{weight}_i Gini(\text{child}_i)$$

$$= G(\text{parent}) - \sum w_i G(\text{child}_i)$$

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

$$Entropy = - \sum_{i=1}^n p_i \log_2(p_i)$$



Gini Criterion & Information Gain

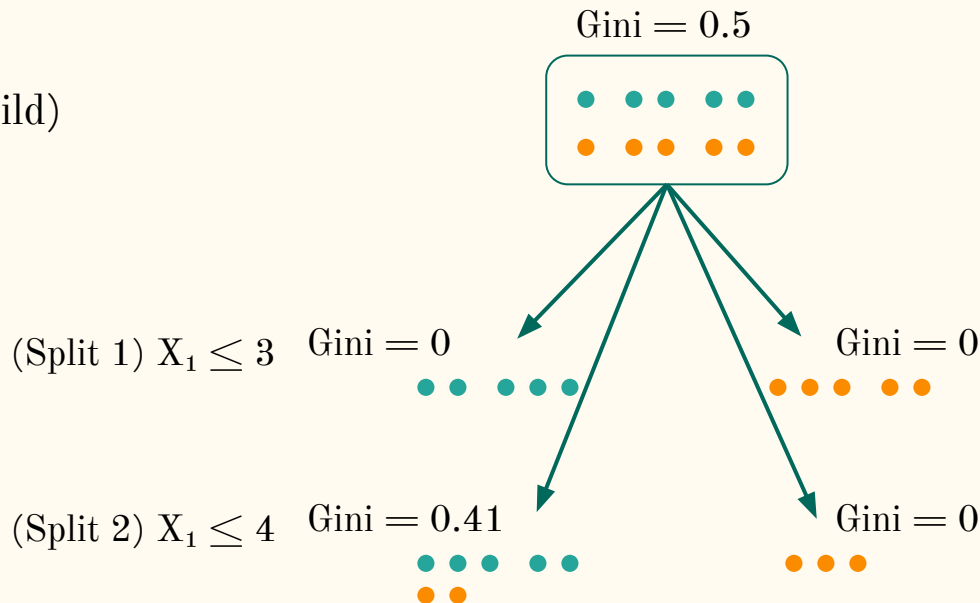
Informarion Gain

$= \text{Information}(\text{parent}) - \text{Information}(\text{child})$

$= Gini(\text{parent}) - \sum \text{weight}_i Gini(\text{child}_i)$

$= G(\text{parent}) - \sum w_i G(\text{child}_i)$

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$



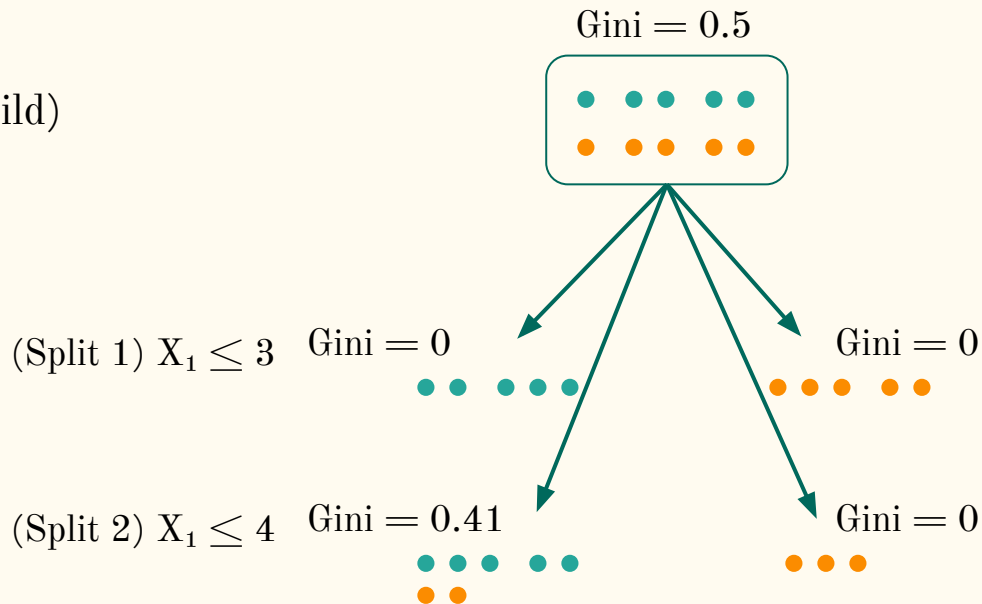
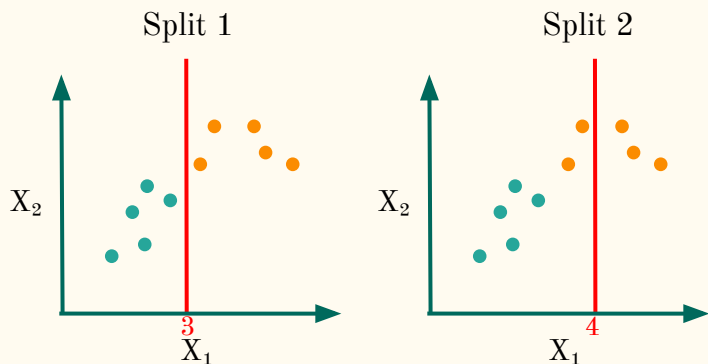
Gini Criterion & Information Gain

Informarion Gain

= Information(parent) - Information(child)

= $Gini(\text{parent}) - \sum weight_i Gini(\text{child}_i)$

= $G(\text{parent}) - \sum w_i G(\text{child}_i)$



Gini Criterion & Information Gain

Informarion Gain

$$= \text{Information}(\text{parent}) - \text{Information}(\text{child})$$

$$= \text{Gini}(\text{parent}) - \sum \text{weight}_i \text{Gini}(\text{child}_i)$$

$$= \mathbf{G}(\text{parent}) - \sum \mathbf{w}_i \mathbf{G}(\text{child}_i)$$

Split 1:

Information Gain

$$= G(\text{parent}) - \sum w_i G(\text{child}_i)$$

$$= 0.5 - (5/10)*0 - (5/10)*0$$

$$= 0.5$$

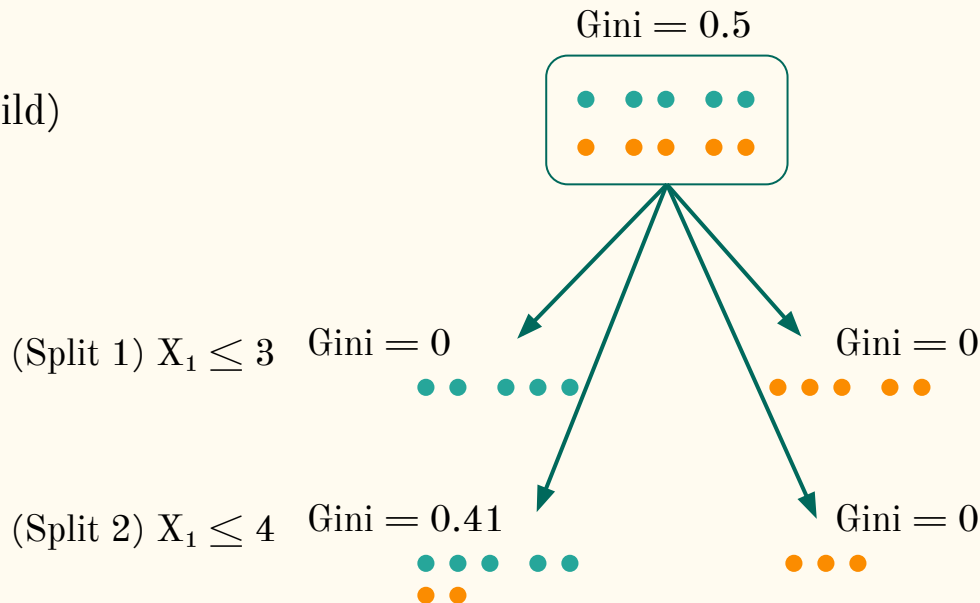
Split 2:

Information Gain

$$= G(\text{parent}) - \sum w_i G(\text{child}_i)$$

$$= 0.5 - (7/10)*0.41 - (3/10)*0$$

$$= 0.213$$



Gini Criterion & Information Gain

Informarion Gain

$$= \text{Information}(\text{parent}) - \text{Information}(\text{child})$$

$$= Gini(\text{parent}) - \sum weight_i Gini(\text{child}_i)$$

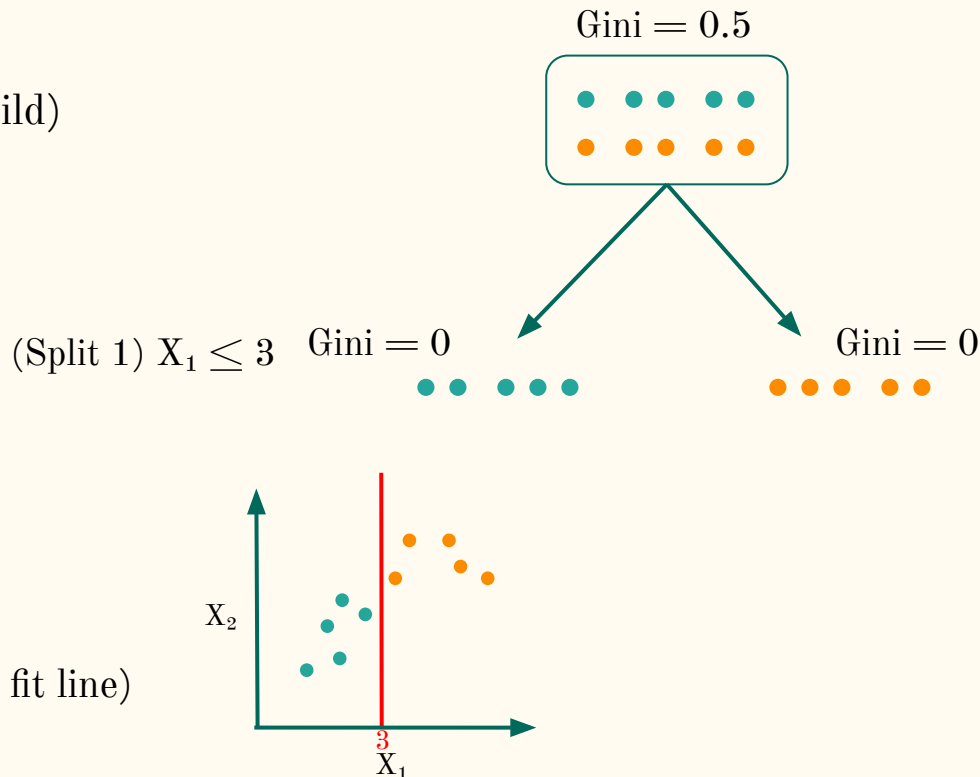
$$= G(\text{parent}) - \sum w_i G(\text{child}_i)$$

$$IG(\text{Split 1}) = 0.5$$

$$IG(\text{Split 2}) = 0.213$$

$$IG(\text{Split 1}) > IG(\text{Split 2})$$

$\Rightarrow$ **Split 1** will be adpoted (as a best fit line)



Inside the blackbox

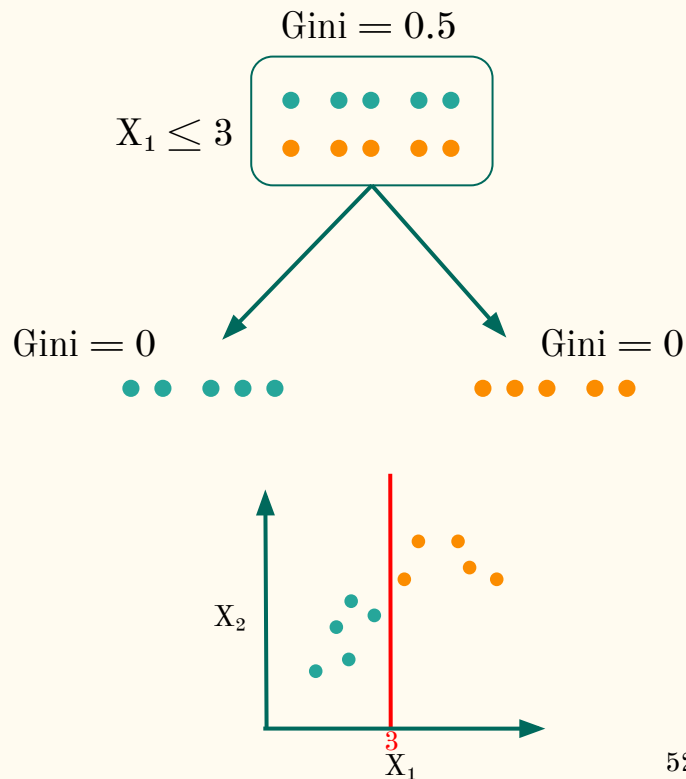
Input $\Rightarrow$ Algos(Decision Tree Classifier) $\Rightarrow$ Output

```
# building the ML algos
Class Decision_Tree_Classifier():
    def __init__(self, tree_depth, ...):
        ...
    def build_tree(data,...):
        # construct the tree
        ...
        return ...

    def gini_index(...):
        # compute gini
        ...
        return ...

    def information_gain(...):
        # compute IG
        ...
        return ...

    def best_split(...):
        ...
        return best_split
```



Inside the blackbox

Input $\Rightarrow$ Algos(Decision Tree Classifier) $\Rightarrow$ Output

```
# building the ML algos
```

```
Class Decision_Tree_Classifier():
```

```
    def __init__(self, tree_depth, ...):
```

```
        ...
```

```
    def build_tree(data,...):
```

```
        # construct the tree
```

```
        ...
```

```
        return ...
```

```
    def gini_index(...):
```

```
        # compute gini
```

```
        ...
```

```
        return ...
```

```
    def information_gain(...):
```

```
        # compute IG
```

```
        ...
```

```
        return
```

```
    def best_split(...):
```

```
        ...
```

```
        return best_split
```

```
    def best_split(...):
```

```
        # find the best split
```

```
        # loop over all features
```

```
        for x in features_list:
```

```
            # loop over all threshold/ feature value in the data
```

```
            for threshold in possible_thresholds:
```

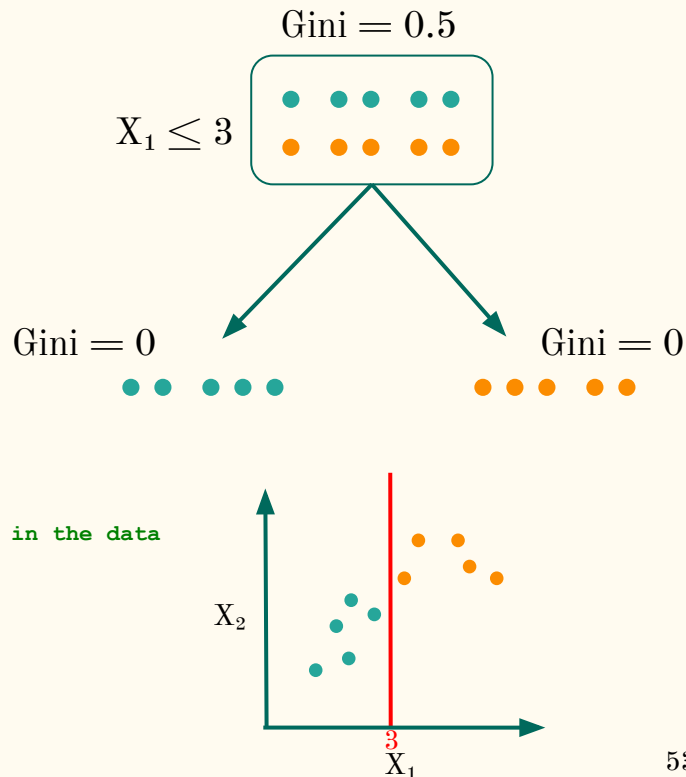
```
                # compute gini
```

```
                # compute IG
```

```
                # compare all splits
```

```
            ...
```

```
        return best_split
```



Machine Learning Framework - Scikit-learn

Input $\Rightarrow$ Algos(Decision Tree Classifier) $\Rightarrow$ Output



```
# building the ML algos
Class Decision_Tree_Classifier():
    def __init__(self, tree_depth, ...):
        ...
    def build_tree(data,...):
        # construct the tree
        ...
        return ...

    def gini_index(...):
        # compute gini
        ...
        return ...

    def information_gain(...):
        # compute IG
        ...
        return ...

    def best_split(...):
        ...
        return best_split
```

```
!pip install scikit-learn
```

```
from sklearn import tree
```

```
clf = tree.DecisionTreeClassifier(max_depth=n)
```

Scikit-learn



```
# only execute once if necessary
```

```
!pip install scikit-learn
```

```
from sklearn.datasets import load_iris
```

```
iris = load_iris() # dictionary
```

```
print(iris)
```

```
{'DESCR': '.. _iris_dataset: ...', 'data':  
array([[5.1, 3.5, 1.4, 0.2],  
       [4.9, 3. , 1.4, 0.2],  
       [4.7, 3.2, 1.3, 0.2],  
       [4.6, 3.1, 1.5, 0.2], ...,  
       [5.9, 3. , 5.1, 1.8]]),  
'data_module': 'sklearn.datasets.data',  
'feature_names': ['sepal length (cm)',  
'sepal width (cm)', 'petal length (cm)',  
'petal width (cm)'], 'filename':  
'iris.csv', 'frame': None, 'target':  
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,  
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
       1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,  
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,  
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,  
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]),  
'target_names': array(['setosa',  
'versicolor', 'virginica'], dtype='<U10')}
```

Iris flower dataset

iris setosa



petal

sepal

iris versicolor



petal

sepal

iris virginica



petal

sepal

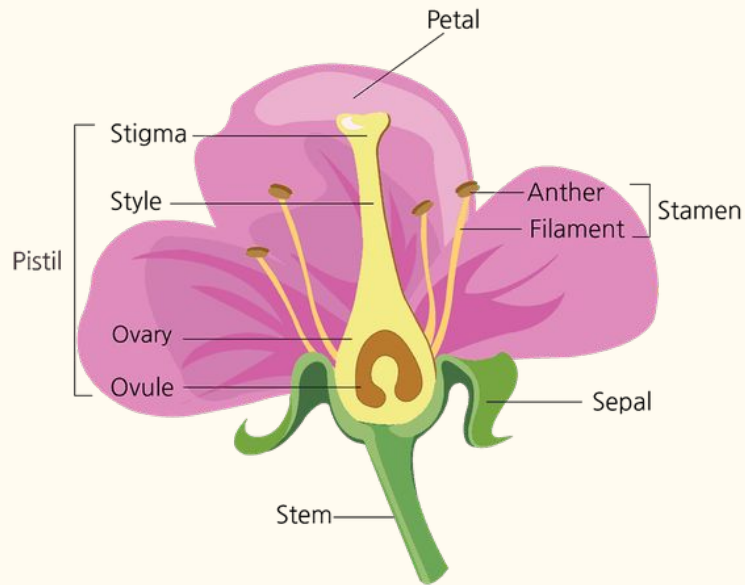
Species:

A

B

C

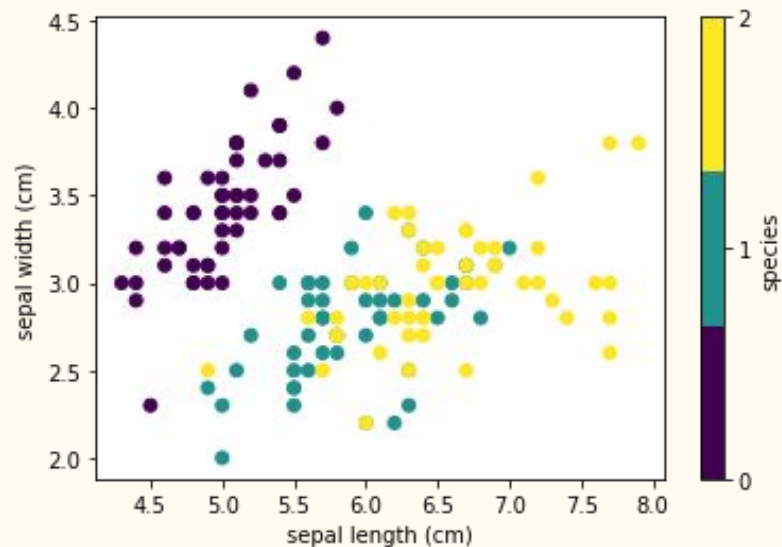
Iris flower dataset



Iris flower dataset

```
import matplotlib.pyplot as plt
x_index = 0 # index of feature
y_index = 1 # index of feature

# make a scatter plot
plt.scatter(iris.data[:,x_index],
            iris.data[:, y_index],
            c=iris.target,cmap=plt.cm.get_cmap(lut=3))
plt.xlabel(iris.feature_names[x_index])
plt.ylabel(iris.feature_names[y_index])
plt.colorbar(ticks=[0,1,2],label="species")
plt.show()
```



Example 1: classify iris setosa(A) vs versicolor(B)

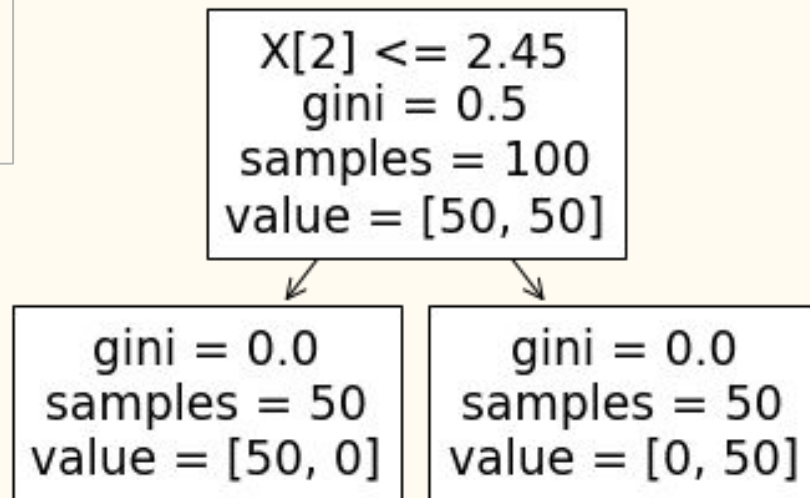
```
from sklearn import tree
# classify iris setosa(A) vs versicolor(B)
X = iris.data[iris.target!=2] # features
y = iris.target[iris.target!=2] # labels
clf = tree.DecisionTreeClassifier(max_depth=2)
clf.fit(X, y)
tree.plot_tree(clf)
```

```
'feature_names': ['sepal length (cm)',
'sepal width (cm)', 'petal length (cm)',
'petal width (cm)']
'target_names': array(['setosa',
'versicolor', 'virginica'], dtype='<U10')
```

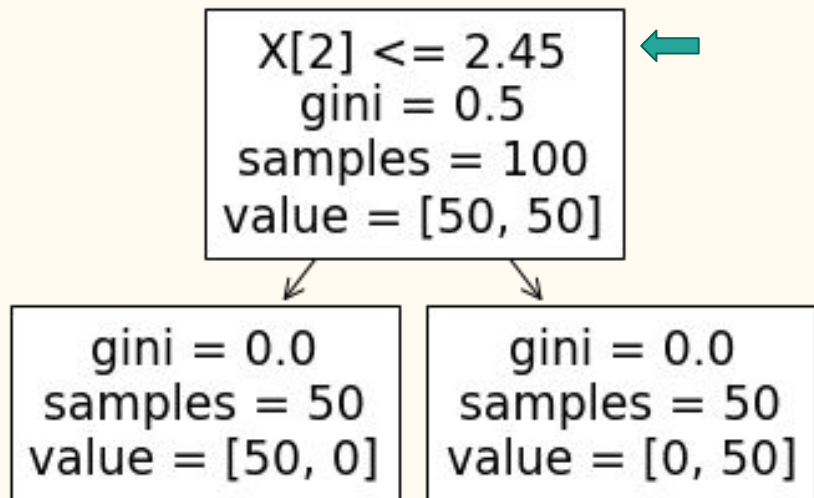
Example 1: classify iris setosa(A) vs versicolor(B)

```
from sklearn import tree
# classify iris setosa(A) vs versicolor(B)
X = iris.data[iris.target!=2] # features
y = iris.target[iris.target!=2] # labels
clf = tree.DecisionTreeClassifier(max_depth=2)
clf.fit(X, y)
tree.plot_tree(clf)
```

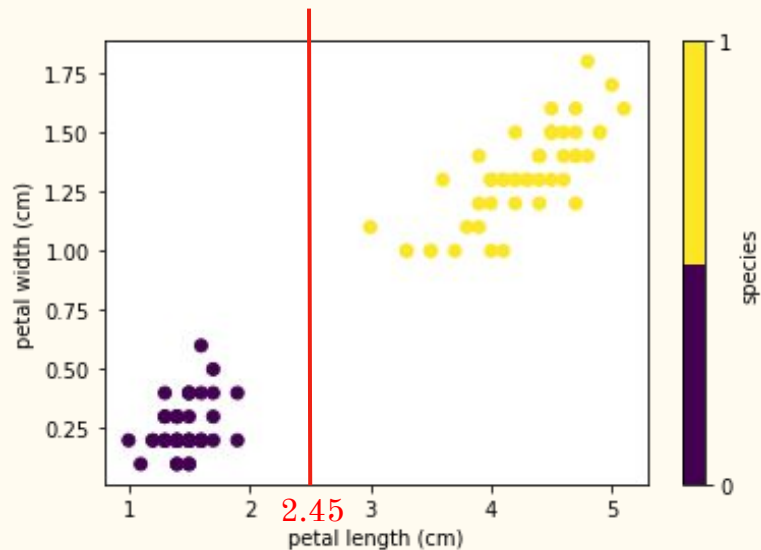
```
'feature_names': ['sepal length (cm)',
'sepal width (cm)', 'petal length (cm)',
'petal width (cm)']
'target_names': array(['setosa',
'versicolor', 'virginica'], dtype='<U10')
```



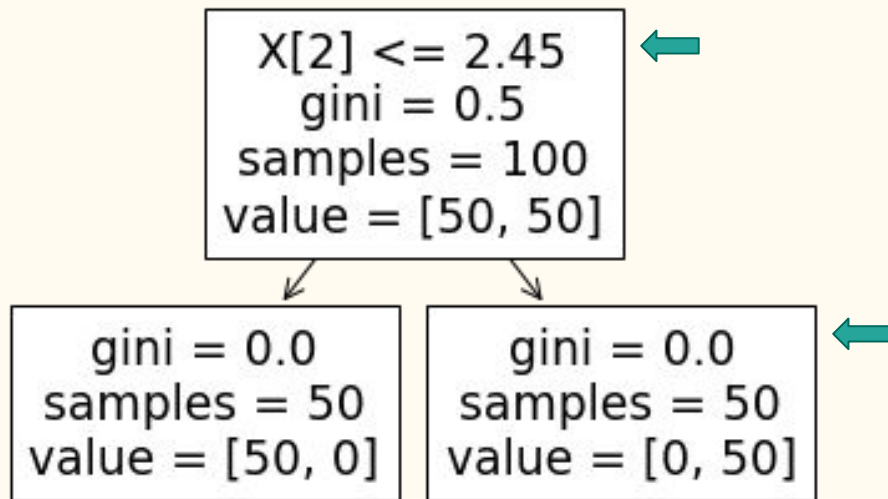
Example 1: classify iris setosa(A) vs versicolor(B)



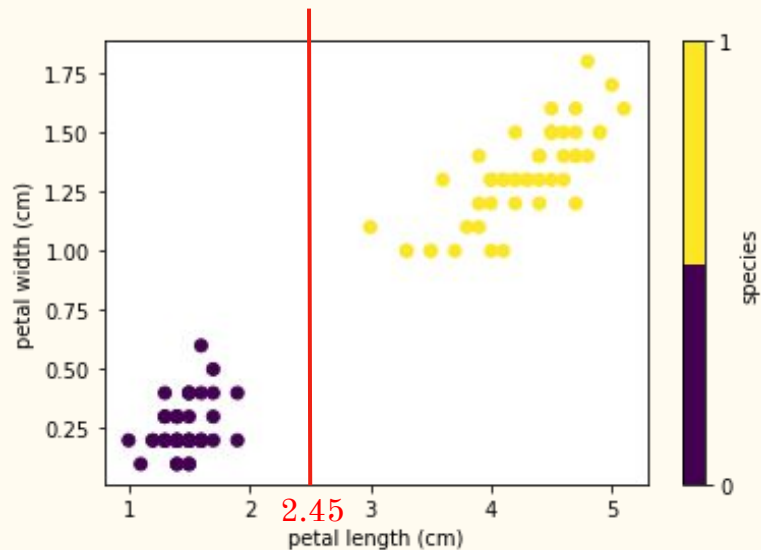
```
'feature_names': ['sepal length (cm)',  
'sepal width (cm)', 'petal length (cm)',  
'petal width (cm)']  
'target_names': array(['setosa',  
'versicolor', 'virginica'], dtype='<U10')
```



Example 1: classify iris setosa(A) vs versicolor(B)



```
'feature_names': ['sepal length (cm)',  
'sepal width (cm)', 'petal length (cm)',  
'petal width (cm)']  
'target_names': array(['setosa',  
'versicolor', 'virginica'], dtype='<U10')
```



Petal length can be used to fully separate the two species: setosa < 2.45cm, versicolor > 2.45cm!

Example 2: classify iris versicolor(B) vs virginica(C)

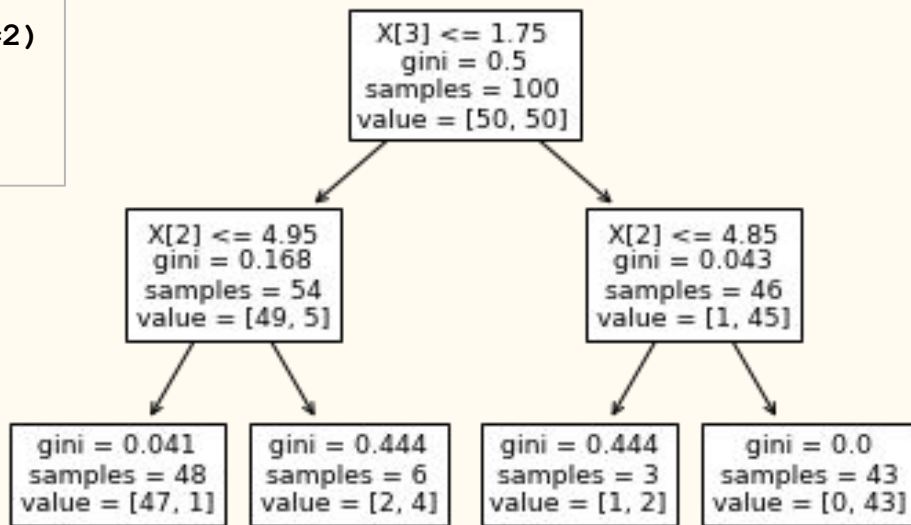
```
from sklearn import tree
# classify iris versicolor(B) vs virginica(C)
X = iris.data[iris.target!=0] # features
y = iris.target[iris.target!=0] # labels
clf = tree.DecisionTreeClassifier(max_depth=2)
clf.fit(X, y)
tree.plot_tree(clf)
```

```
'feature_names': ['sepal length (cm)',
'sepal width (cm)', 'petal length (cm)',
'petal width (cm)']
'target_names': array(['setosa',
'versicolor', 'virginica'], dtype='<U10')
```

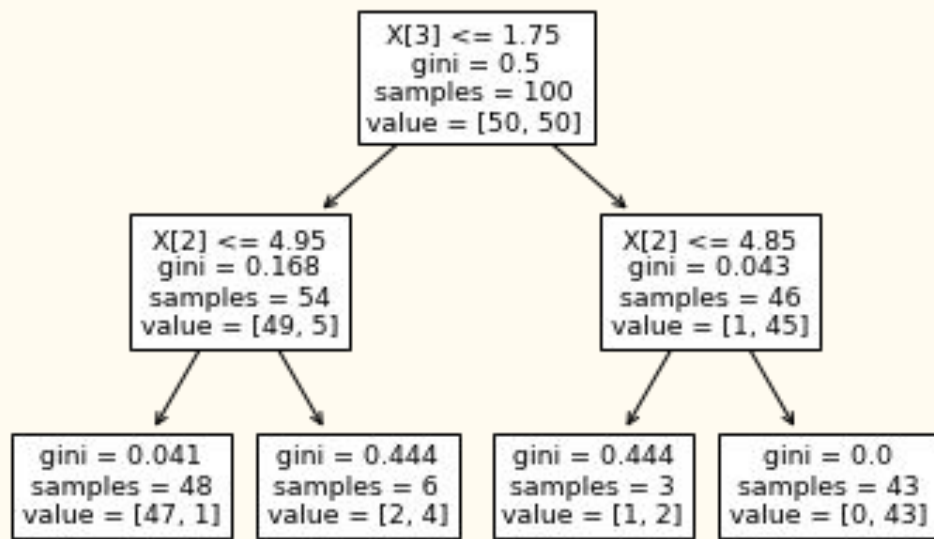
Example 2: classify iris versicolor(B) vs virginica(C)

```
from sklearn import tree
# classify iris versicolor(B) vs virginica(C)
X = iris.data[iris.target!=0] # features
y = iris.target[iris.target!=0] # labels
clf = tree.DecisionTreeClassifier(max_depth=2)
clf.fit(X, y)
tree.plot_tree(clf)
```

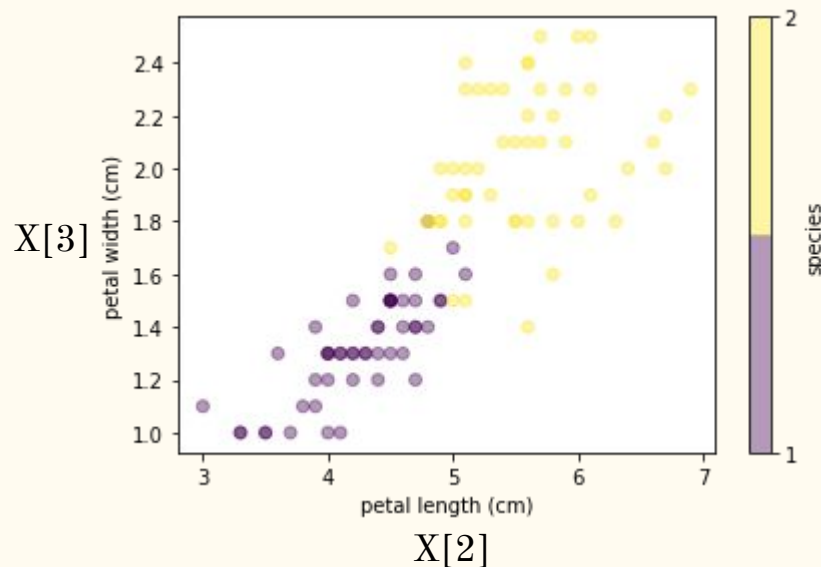
```
'feature_names': ['sepal length (cm)',
'sepal width (cm)', 'petal length (cm)',
'petal width (cm)']
'target_names': array(['setosa',
'versicolor', 'virginica'], dtype='<U10')
```



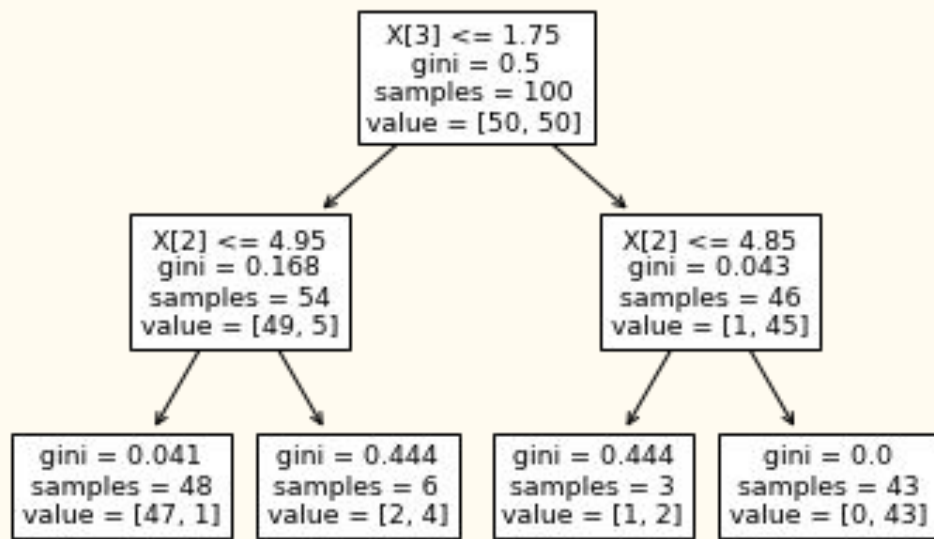
Example 2: classify iris versicolor(B) vs virginica(C)



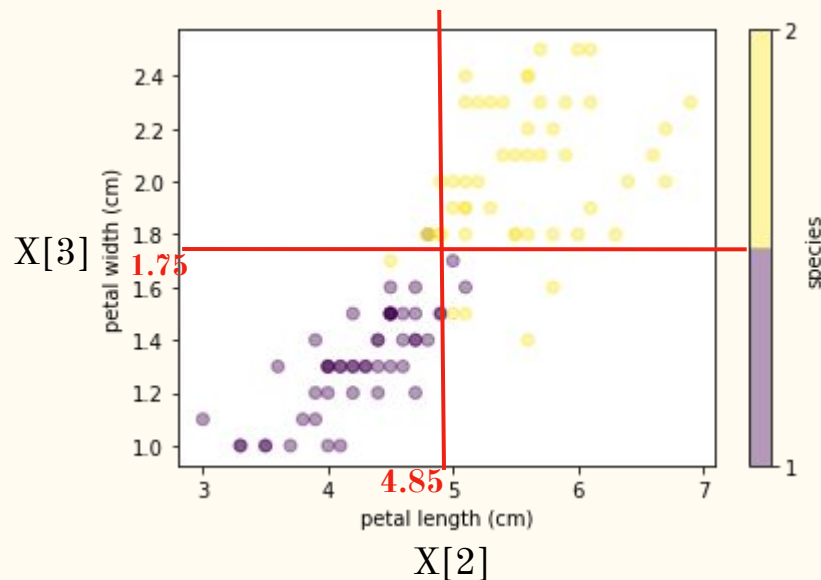
```
'feature_names': ['sepal length (cm)',  
'sepal width (cm)', 'petal length (cm)',  
'petal width (cm)']  
'target_names': array(['setosa',  
'versicolor', 'virginica'], dtype='<U10')
```



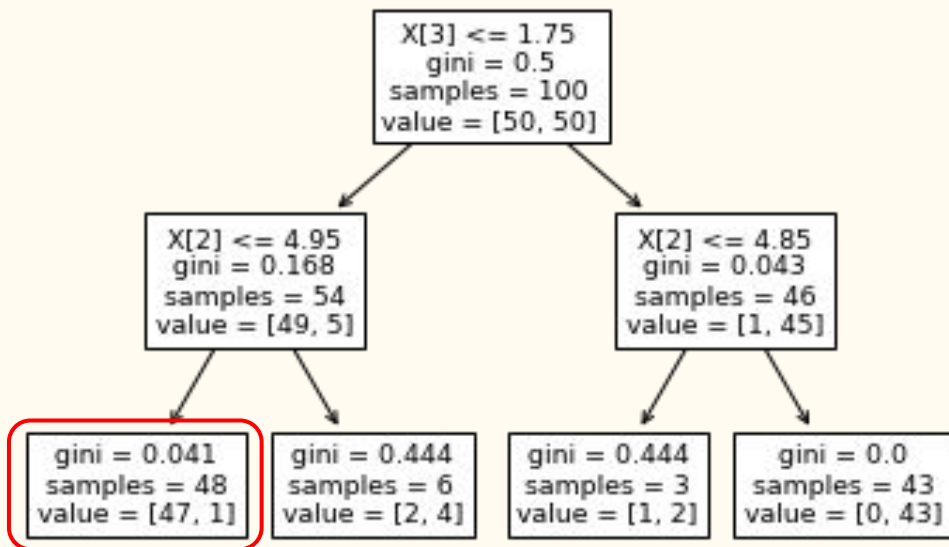
Example 2: classify iris versicolor(B) vs virginica(C)



```
'feature_names': ['sepal length (cm)',  
'sepal width (cm)', 'petal length (cm)',  
'petal width (cm)']  
'target_names': array(['setosa',  
'versicolor', 'virginica'], dtype='<U10')
```



Example 2: classify iris versicolor(B) vs virginica(C)



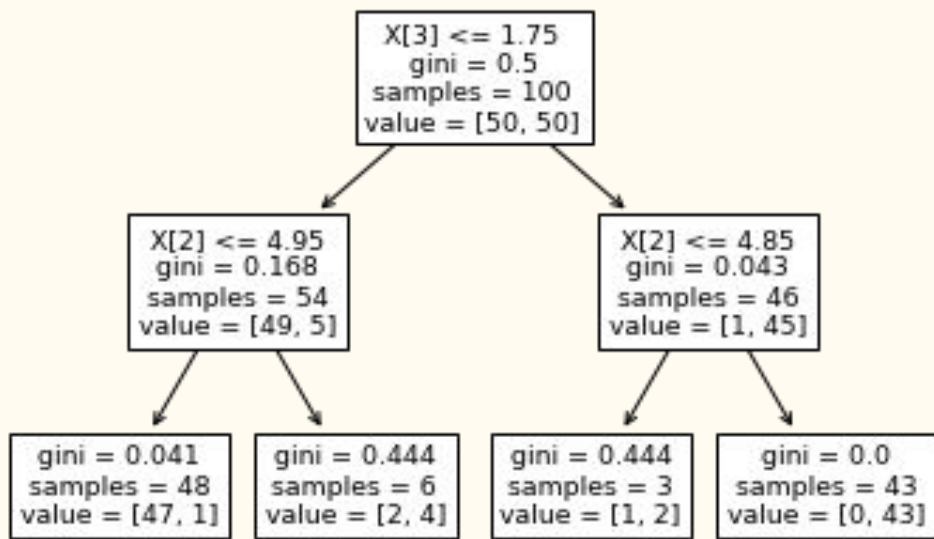
$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

P(Species = B): 47/48

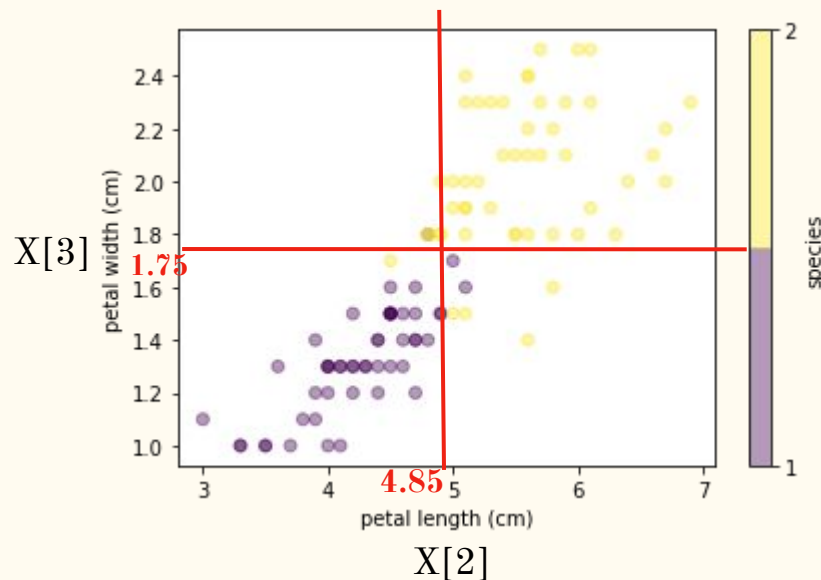
P(Species = C): 1/48

$$\begin{aligned} \text{Gini index} &= 1 - [P(B)^2 + P(C)^2] \\ &= 0.0407... \\ &\sim 0.041 \end{aligned}$$

Example 2: classify iris versicolor(B) vs virginica(C)



```
'feature_names': ['sepal length (cm)',  
'sepal width (cm)', 'petal length (cm)',  
'petal width (cm)']  
'target_names': array(['setosa',  
'versicolor', 'virginica'], dtype='<U10')
```



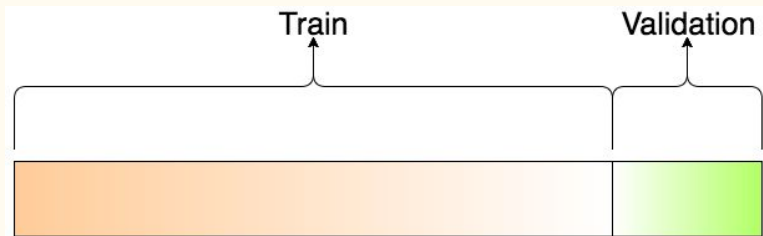
In comparison, tree depth = 2 is still insufficient to fully separate iris versicolor vs virginica!

Scikit-learn: train test split

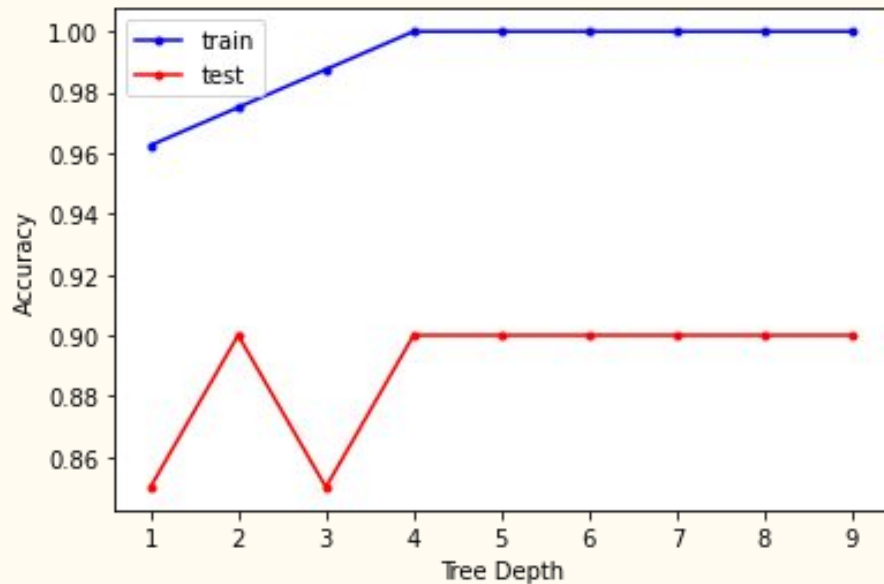
```
from sklearn.model_selection import train_test_split
# classify iris versicolor vs virginica
X = iris.data[iris.target!=0] # features
y = iris.target[iris.target!=0] # labels

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2) # set aside 20% of data for validation
print(len(X_train))
print(len(X_test))
```

80
20



Scikit-learn: optimal tree depth



```
score_test = [] # create list to store the scores
score_train = []
for i in np.arange(1,10):
    clf = tree.DecisionTreeClassifier(max_depth=i)
    clf.fit(X_train, y_train)
    score_train += [clf.score(X_train, y_train)]
    score_test += [clf.score(X_test, y_test)]

plt.plot(np.arange(1,10), score_train, "b.-", label="train")
plt.plot(np.arange(1,10), score_test, "r.-", label="test")
plt.xlabel("Tree Depth")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

Metrics - Confusion matrix

TP True Positive

TN True Negative

FP False Positive

FN False Negative

		Predicted classes		Actual Classes	Predicted Classes	
		Class 1	Class 2			
Actual classes	Class 1	TP	FN	1	1	TP
				1	1	TP
	Class 2			1	2	FN
		FP	TN	2	2	TN
				2	1	FP
				2	2	TN
				...	...	...

Metrics - Accuracy

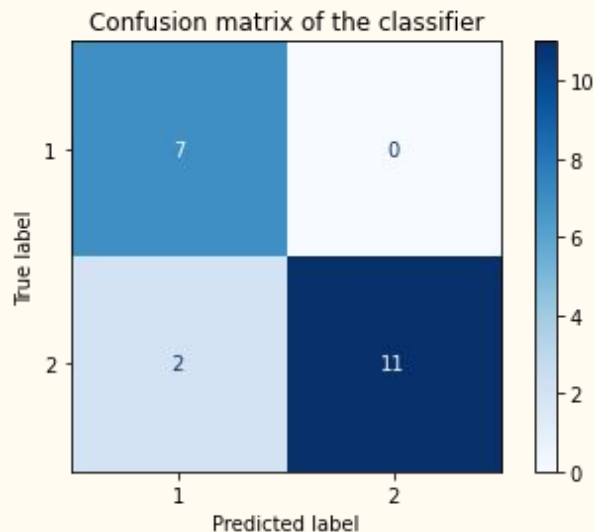
$$Accuracy = \frac{\text{Correct Predictions}}{\text{All Predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

		Predicted classes		Actual Classes	Predicted Classes		
		Class 1	Class 2				
Actual classes	Class 1	2	1	1	1	TP	Accuracy = (2+2)/(2+2+1+1) = 4/6 = 0.667
				1	1	TP	
				1	2	FN	
				2	2	TN	
Class 2		1	2	2	1	FP	
				2	2	TN	

Metrics - Accuracy (Iris flower example)

		Predicted classes	
		versicolor(B)	virginica(C)
Actual classes	versicolor(B)	?	?
	virginica(C)	?	?

Metrics - Accuracy (Iris flower example)



```
> Predicted classes:  
[2 2 1 1 1 2 1 2 2 2 2 1 1 2 1 1 2 1 2 2]
```

```
> Actual classes:  
[2 2 2 1 1 2 1 2 2 2 2 1 2 2 1 1 2 1 2 2]
```

```
> Accuracy: 0.9
```

```
from sklearn.metrics import accuracy_score, plot_confusion_matrix  
  
# prediction on test dataset  
y_pred = clf.predict(X_test)  
# compare predicted classes and actual classes  
print("Predicted classes:\n", y_pred)  
print("Actual classes:\n", y_test)  
  
labels=[1,2] # correspond to "versicolor" (1) and "virginica" (2)  
cm = plot_confusion_matrix(clf, X_test,  
                           y_test, labels=labels, cmap="Blues")  
plt.title("Confusion matrix of the classifier")  
plt.show()  
  
ac = accuracy_score(y_test, y_pred)  
print(f"Accuracy: {ac}")
```

Metrics (Classification Context)

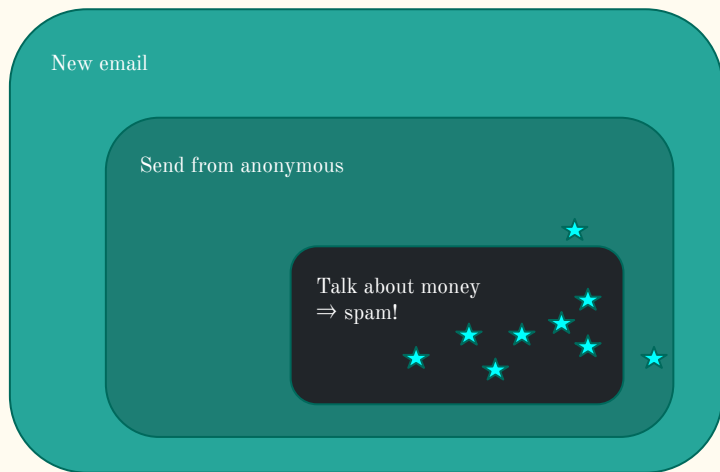
Commonly used metrics: Accuracy, Precision, Recall, F1-score

Sources: [5][6][7][8][9][10][11][12] view · talk · edit

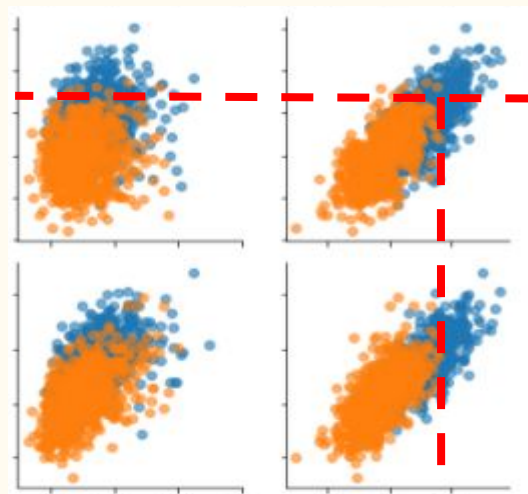
		Predicted condition			
		Positive (PP)	Negative (PN)	Informedness, bookmaker informedness (BM) $= \text{TPR} + \text{TNR} - 1$	Prevalence threshold (PT) $= \frac{\sqrt{\text{TPR} \times \text{FPR}} - \text{FPR}}{\text{TPR} - \text{FPR}}$
Actual condition	Total population $= P + N$				
	Positive (P)	True positive (TP), hit	False negative (FN), type II error, miss, underestimation	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power $= \frac{\text{TP}}{P} = 1 - \text{FNR}$	False negative rate (FNR), miss rate $= \frac{\text{FN}}{P} = 1 - \text{TPR}$
	Negative (N)	False positive (FP), type I error, false alarm, overestimation	True negative (TN), correct rejection	False positive rate (FPR), probability of false alarm, fall-out $= \frac{\text{FP}}{N} = 1 - \text{TNR}$	True negative rate (TNR), specificity (SPC), selectivity $= \frac{\text{TN}}{N} = 1 - \text{FPR}$
	Prevalence $= \frac{P}{P + N}$	Positive predictive value (PPV), precision $= \frac{\text{TP}}{\text{PP}} = 1 - \text{FDR}$	False omission rate (FOR) $= \frac{\text{FN}}{\text{PN}} = 1 - \text{NPV}$	Positive likelihood ratio (LR+) $= \frac{\text{TPR}}{\text{FPR}}$	Negative likelihood ratio (LR-) $= \frac{\text{FNR}}{\text{TNR}}$
	Accuracy (ACC) $= \frac{\text{TP} + \text{TN}}{P + N}$	False discovery rate (FDR) $= \frac{\text{FP}}{\text{PP}} = 1 - \text{PPV}$	Negative predictive value (NPV) $= \frac{\text{TN}}{\text{PN}} = 1 - \text{FOR}$	Markedness (MK), deltaP (Δp) $= \text{PPV} + \text{NPV} - 1$	Diagnostic odds ratio (DOR) $= \frac{\text{LR}+}{\text{LR}-}$
	Balanced accuracy (BA) $= \frac{\text{TPR} + \text{TNR}}{2}$	F ₁ score $= \frac{2\text{PPV} \times \text{TPR}}{\text{PPV} + \text{TPR}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$	Fowlkes–Mallows index (FM) $= \sqrt{\text{PPV} \times \text{TPR}}$	Matthews correlation coefficient (MCC) $= \sqrt{\text{TPR} \times \text{TNR} \times \text{PPV} \times \text{NPV} - \sqrt{\text{FNR} \times \text{FPR} \times \text{FOR} \times \text{FDR}}}$	Threat score (TS), critical success index (CSI), Jaccard index $= \frac{\text{TP}}{\text{TP} + \text{FN} + \text{FP}}$

Where we are so far

Rule-based approach



Machine learning approach



Machine learning approach

Supervised Learning

Regression

Linear Regression

Decision Tree

Random Forest

Neural Networks

Classification

Logistic Regression

Naive Bayes

DT

RF

NN

Support Vector Machines

Unsupervised Learning

Clustering

Centroid

Distribution

Density

Hierarchical

Dimensionality Reduction

Principal Component Analysis (PCA)

Machine learning approach

Supervised Learning

Regression

Linear Regression

Decision Tree

Random Forest

Neural Networks

Classification

Logistic Regression

Naive Bayes

DT

RF

NN

Support Vector Machines

Unsupervised Learning

Clustering

Centroid

Distribution

Density

Hierarchical

Dimensionality Reduction

Principal Component Analysis (PCA)

Machine learning approach

Supervised Learning

Regression

Linear Regression

Decision Tree

Random Forest

Neural Networks

Classification

Logistic Regression

Naive Bayes

DT

RF

NN

Support Vector Machines

Unsupervised Learning

Clustering

Centroid

Distribution

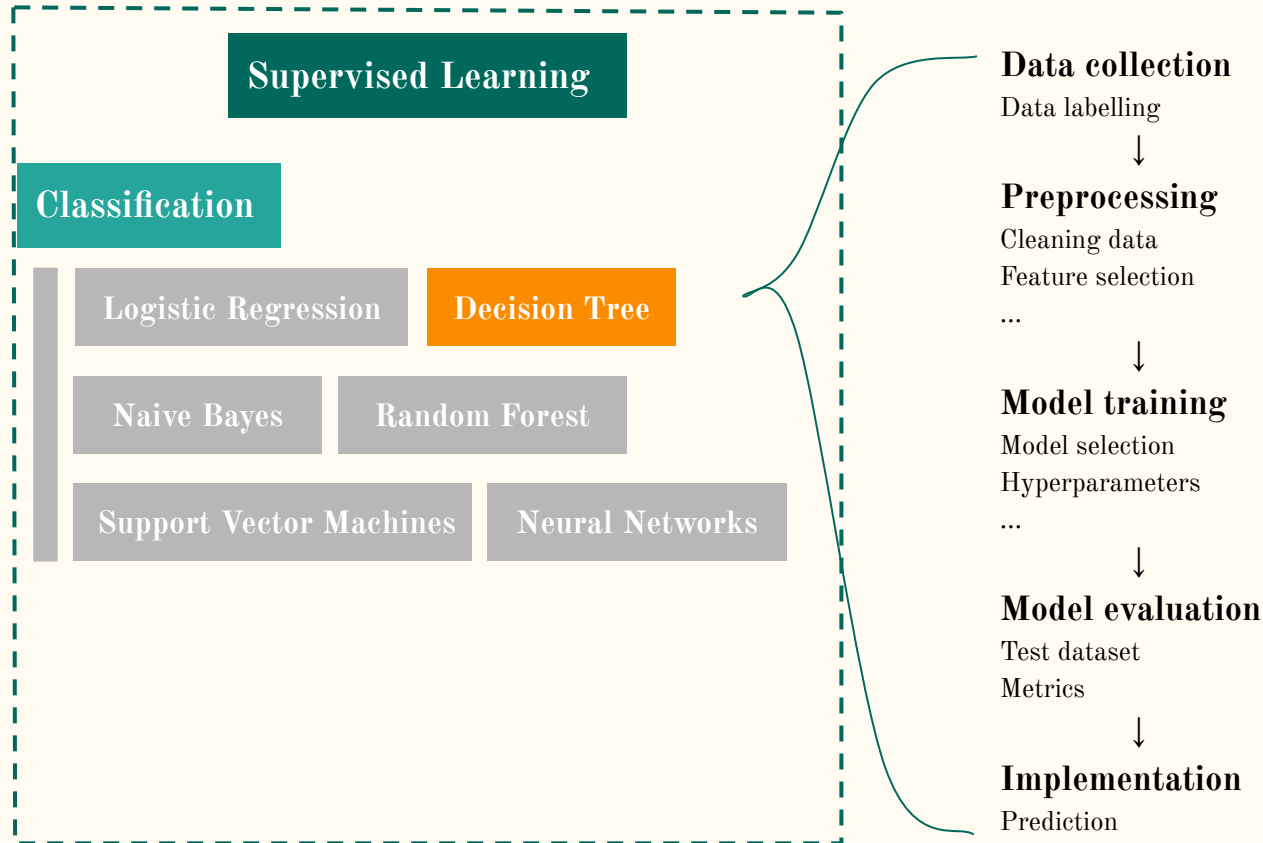
Density

Hierarchical

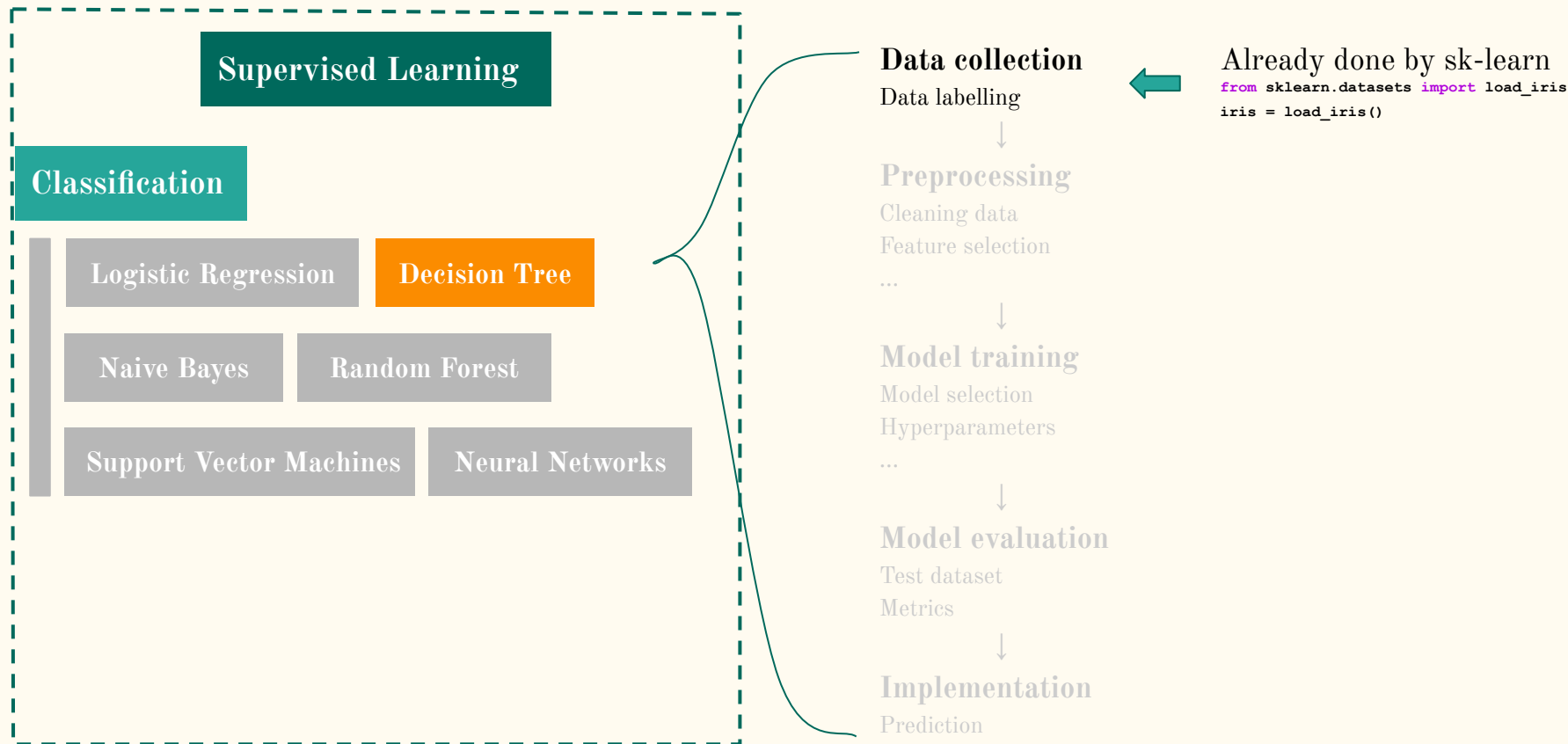
Dimensionality Reduction

Principal Component Analysis (PCA)

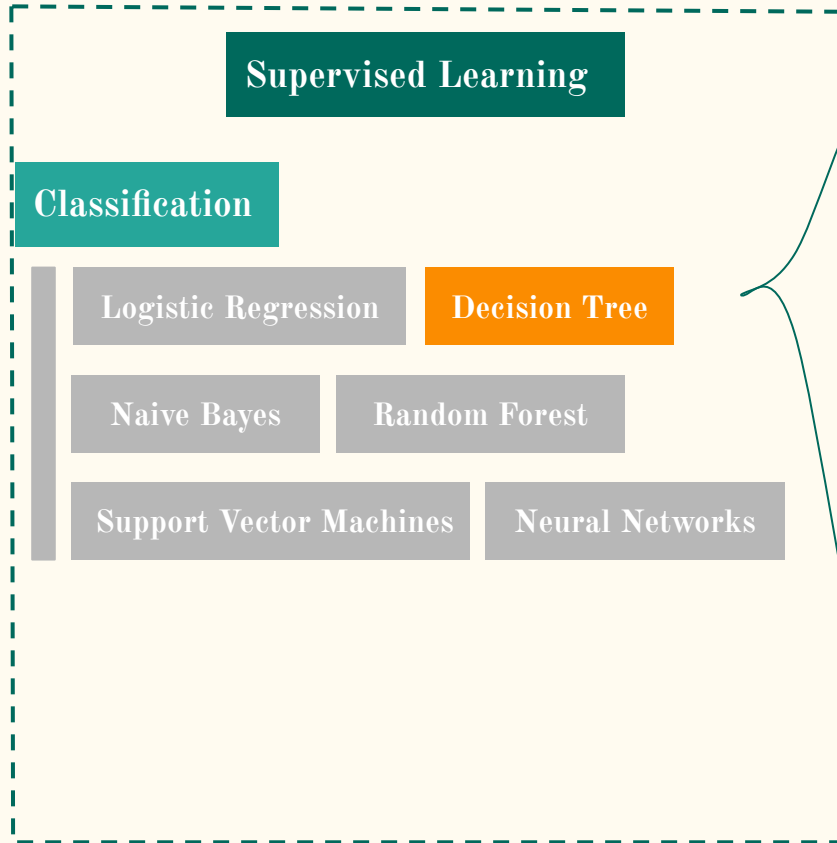
Machine learning approach



Machine learning approach



Machine learning approach



Data collection

Data labelling



Preprocessing

Cleaning data

Feature selection

...



Model training

Model selection

Hyperparameters

...



Model evaluation

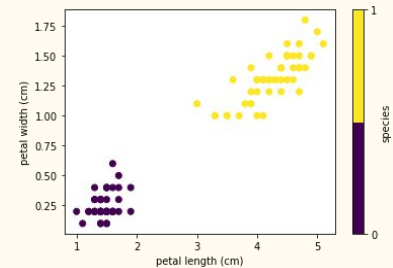
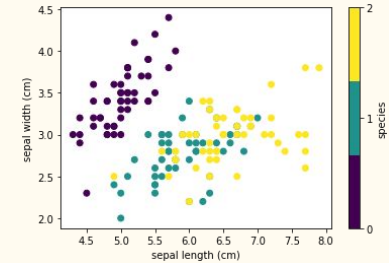
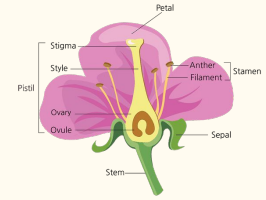
Test dataset

Metrics

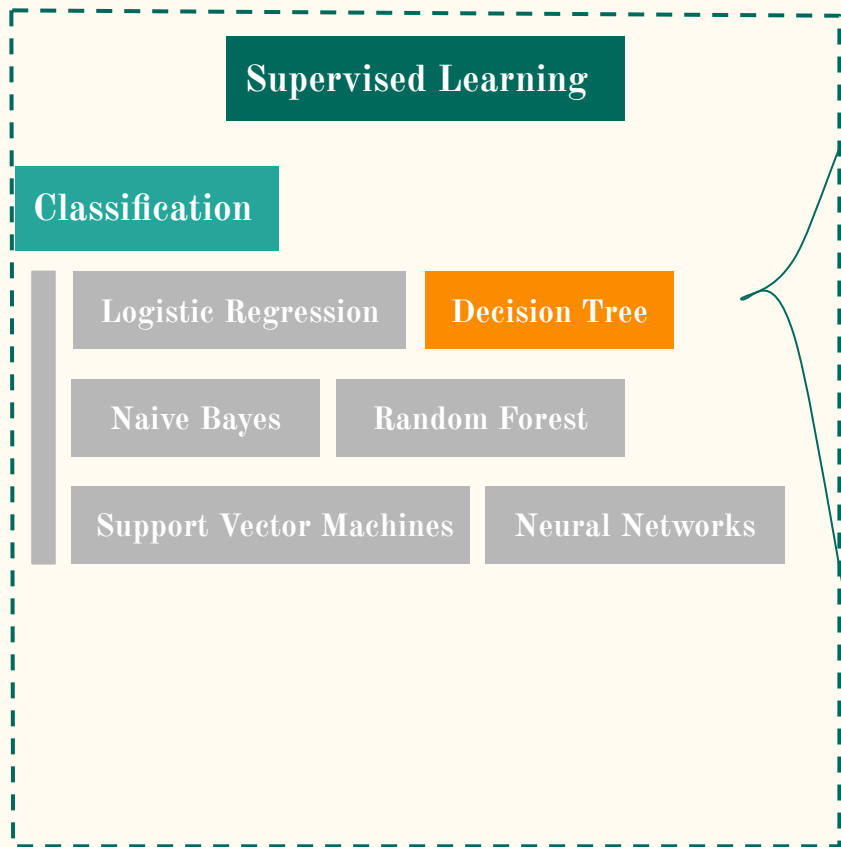


Implementation

Prediction



Machine learning approach



Data collection

Data labelling



Preprocessing

Cleaning data

Feature selection

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics

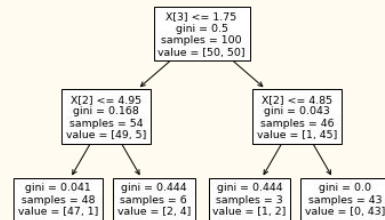


Implementation

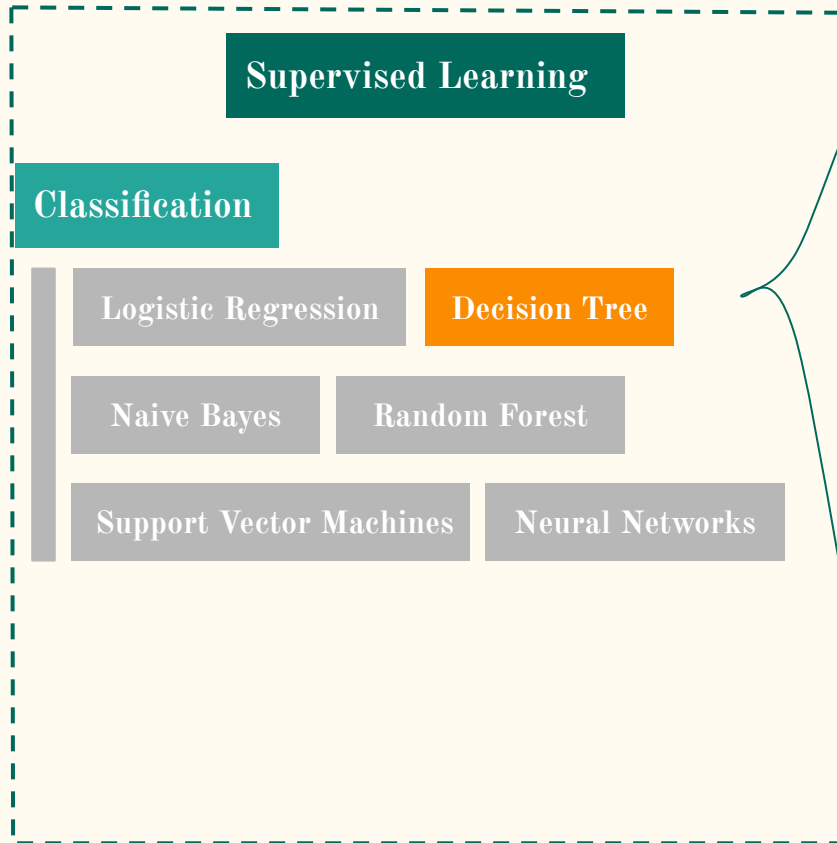
Prediction

Using **decision tree classifier**

Using validation set to tune the hyperparameters: **tree depth n**



Machine learning approach



Data collection

Data labelling



Preprocessing

Cleaning data

Feature selection

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

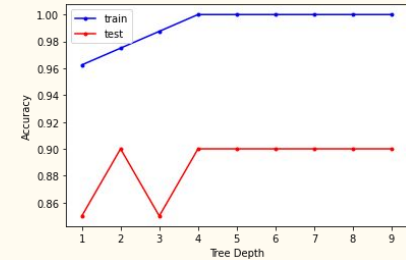
Metrics



Implementation

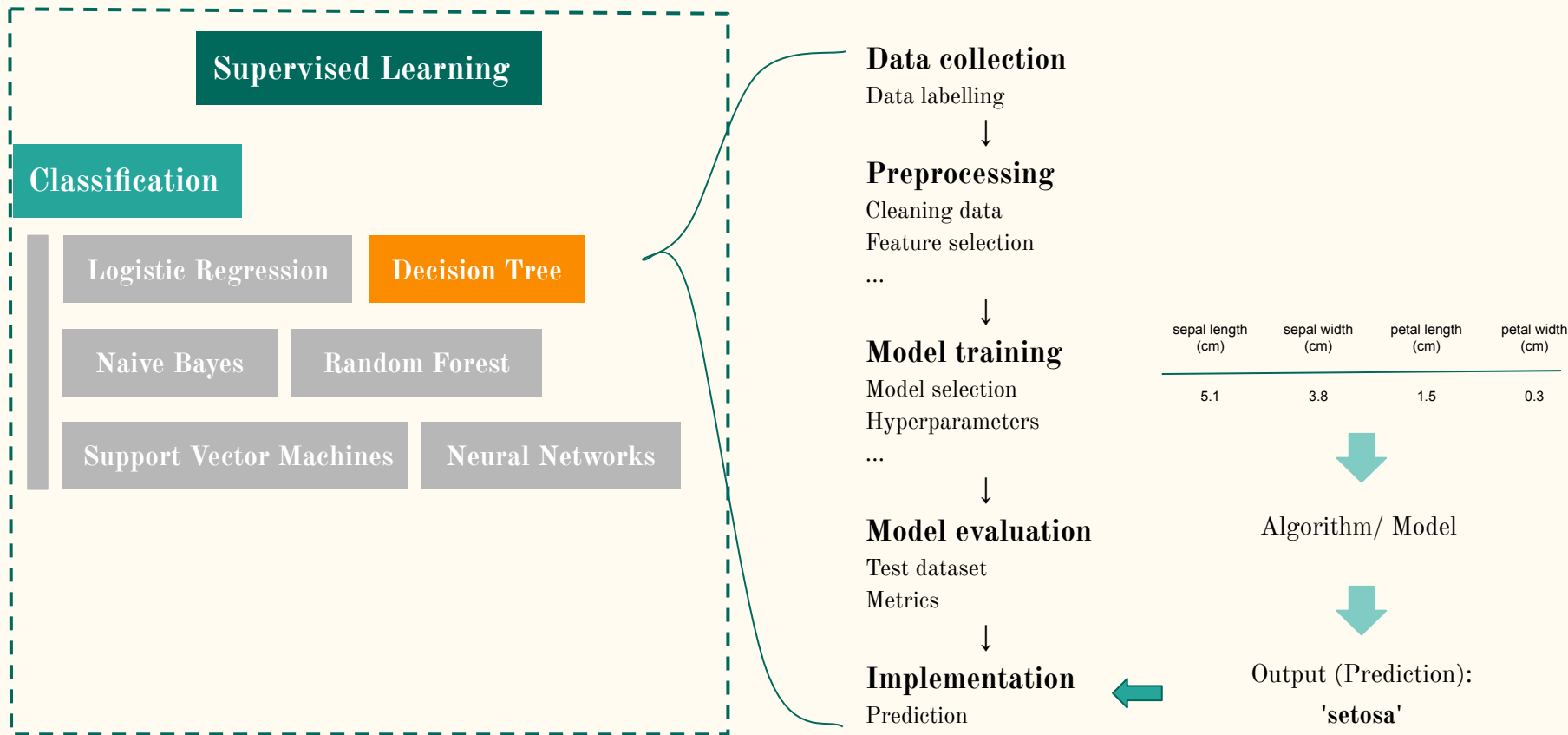
Prediction

Using metrics to measure the model **performance** & finalize the model

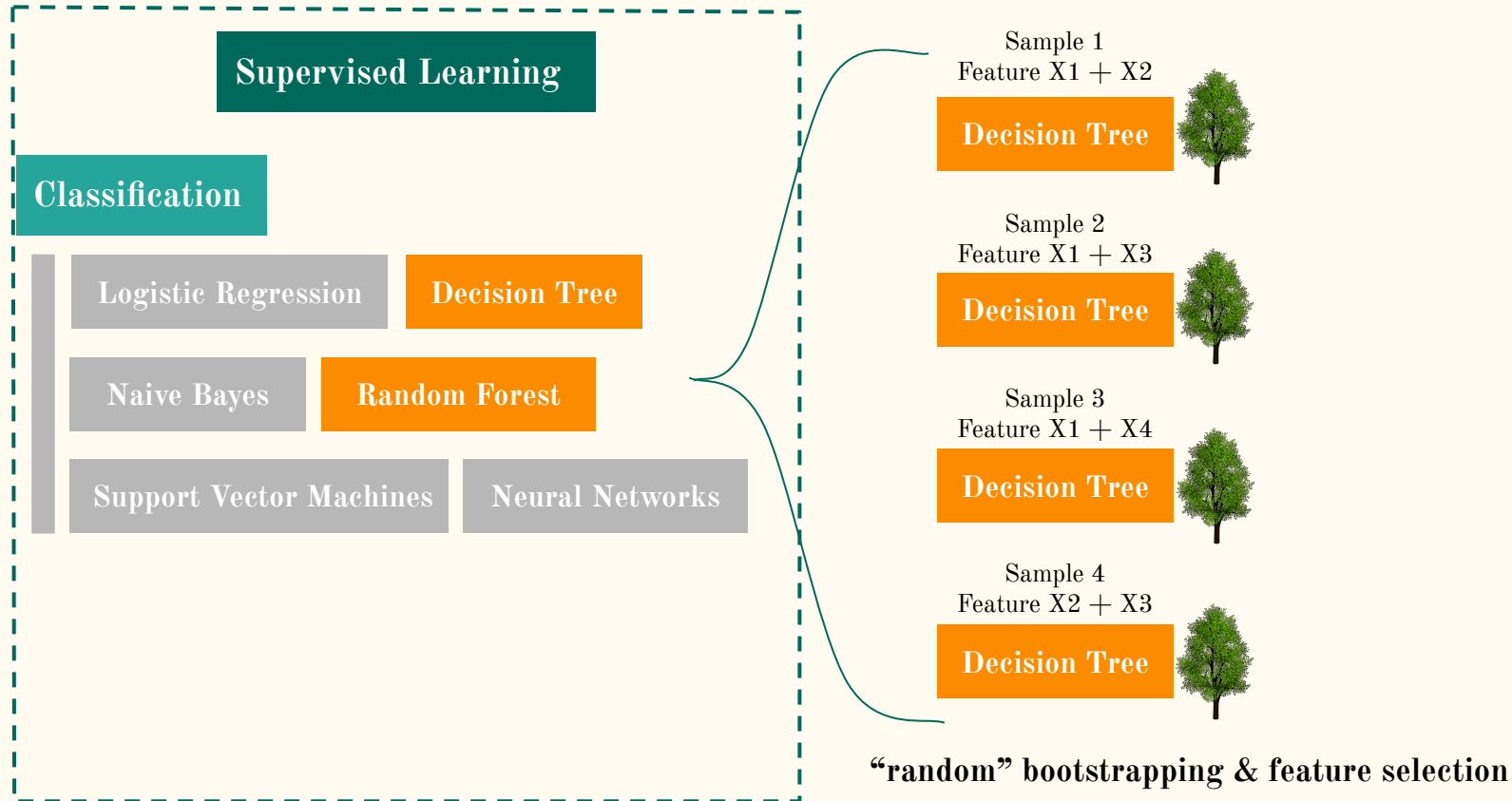


	versicolor(B)	virginica(C)
versicolor(B)	TP	FN
virginica(C)	FP	TN

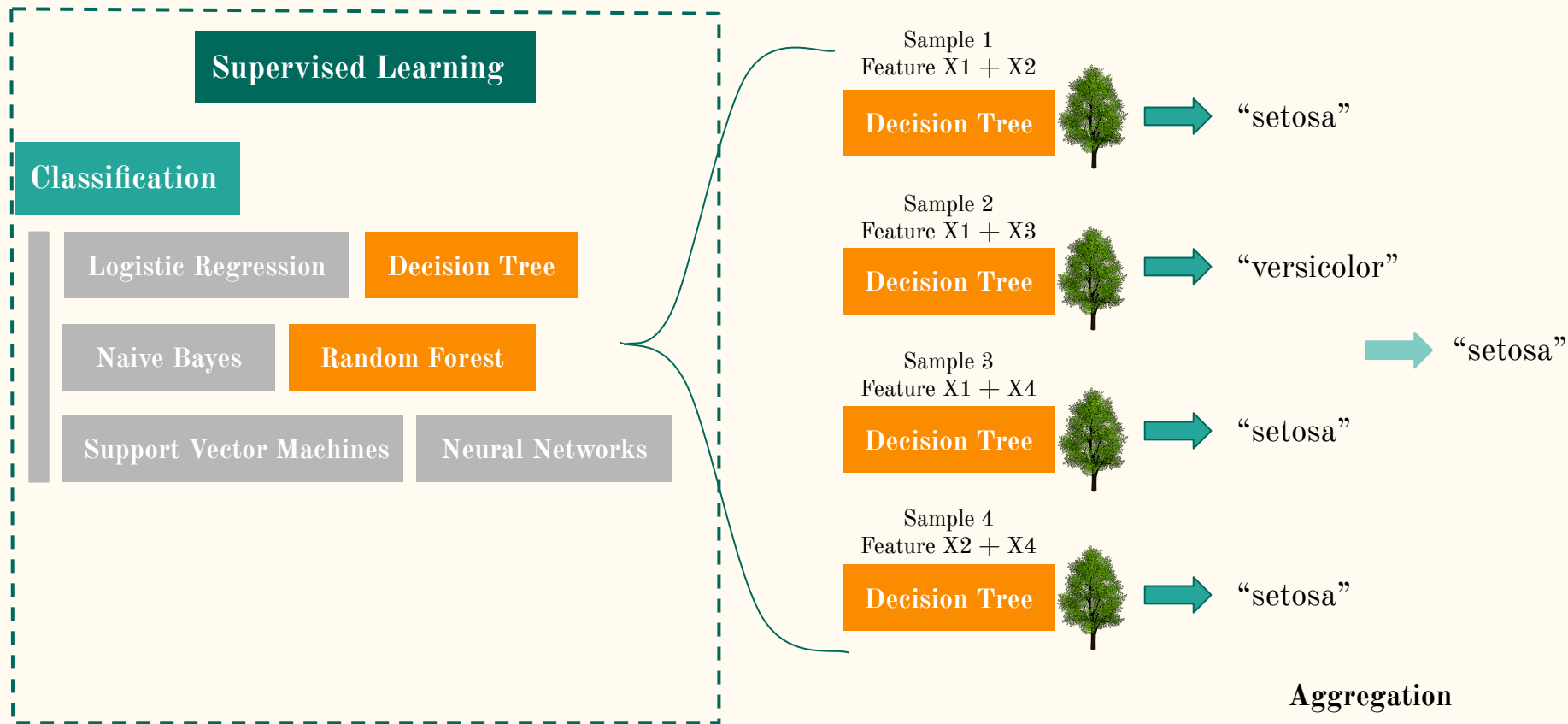
Machine learning approach



Machine learning approach



Machine learning approach



This workshop will cover :

Supervised Learning

Regression

Linear Regression

Decision Tree

Random Forest

Neural Networks

Classification

Logistic Regression

Naive Bayes

DT

RF

NN

Support Vector Machines

Unsupervised Learning

Clustering

Centroid

Distribution

Density

Hierarchical

Dimensionality Reduction

Principal Component Analysis (PCA)

Class Exercise 1:

A) Review on plotting: Apart from petal length, is there **any other feature** that can also be used to fully separate iris setosa(A) vs versicolor(B)? Please make a **scatter plot** to show that.

B) Train a decision tree to classify iris setosa vs versicolor vs virginica. Can all three species **be fully separated**? What's the **optimal tree depth**? Please plot the “tree” for the optimal tree depth.

C) Evaluate the classifier by using **confusion matrix** and **accuracy**. Given an iris flower has a **feature array [5.1, 3.8, 1.5, 0.3]**, make a prediction using the trained classifier.

The first step to machine learning - data collection

This is an anonymous questionnaire to collect some data for the class practice in the next class.

Questionnaire link: <https://forms.gle/34XfAeW7MbX2CbTd9>

Thank You!

Next Lesson

- Supervised: Regression
- Unsupervised: Clustering

